US012418417B1

(12) **United States Patent**
Chen

(10) **Patent No.: US 12,418,417 B1**
(45) **Date of Patent: Sep. 16, 2025**

(54) **BLOCKCHAIN-BASED ARTIFICIAL INTELLIGENCE AGENT LIFE CYCLE MANAGEMENT AND AUTHENTICATION SYSTEMS AND METHODS**

(71) Applicant: **TYNTRE, LLC**, Irvine, CA (US)

(72) Inventor: **Thomas Chen**, Irvine, CA (US)

(73) Assignee: **TYNTRE, LLC**, Irvine, CA (US)

(*) Notice: Subject to any disclaimer, the term of this patent is extended or adjusted under 35 U.S.C. 154(b) by 0 days.

(21) Appl. No.: **19/070,298**

(22) Filed: **Mar. 4, 2025**

(51) **Int. Cl.**
H04L 9/32 (2006.01)
H04L 9/00 (2022.01)

(52) **U.S. Cl.**
CPC ***H04L 9/3213*** (2013.01); ***H04L 9/3218*** (2013.01); ***H04L 9/50*** (2022.05)

(58) **Field of Classification Search**
None
See application file for complete search history.

(56) **References Cited**

U.S. PATENT DOCUMENTS

6,144,739	A *	11/2000	Witt	H04L 63/123 380/278
10,425,414	B1 *	9/2019	Buckingham	G06F 21/43
12,321,446	B1 *	6/2025	Chan	G06F 21/55
2004/0015719	A1 *	1/2004	Lee	H04L 63/0227 709/224
2016/0300070	A1 *	10/2016	Durham	H04L 63/20
2019/0160660	A1 *	5/2019	Husain	B25J 9/1694
2020/0068013	A1 *	2/2020	Zakharov	G06F 21/78
2021/0035018	A1 *	2/2021	Kim	H04L 9/50
2022/0085980	A1 *	3/2022	Muthukrishnan	H04L 9/30
2022/0210142	A1 *	6/2022	Sohail	G06F 21/30
2023/0319037	A1 *	10/2023	Zaman	H04L 63/0876 726/6
2025/0103736	A1 *	3/2025	Shionoiri	G06F 21/6209
2025/0165296	A1 *	5/2025	Hwang	G06F 9/5027

FOREIGN PATENT DOCUMENTS

WO	WO-2023069062	A1 *	4/2023	
WO	WO-2024147292	A1 *	7/2024	G06F 21/62

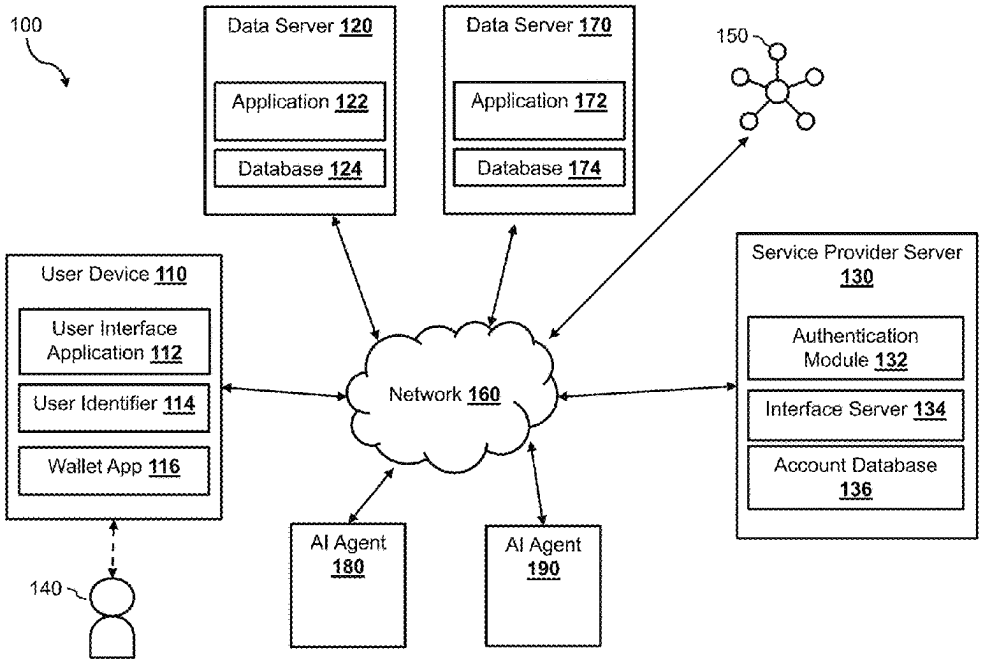
* cited by examiner

Primary Examiner — Taghi T Arani
Assistant Examiner — Joshua Raymond White
(74) *Attorney, Agent, or Firm* — Haynes and Boone, LLP

(57) **ABSTRACT**

Methods and systems are presented for using blockchain technologies to manage the life cycle and authentication of artificial intelligence (AI) agents. When an AI agent is created, the AI agent registers itself with an authentication system. The authentication system creates identity information for the AI agent and store on a blockchain. The identity information is used to track the changes of the AI agent through its life cycle, including upgrading of the binary code, transfer of ownership, change of a delegator, and others. When the AI agent requests for access of one or more resources, the authentication system uses the information stored on the blockchain to authenticate the AI agent before granting the AI agent access to the one or more resources.

20 Claims, 10 Drawing Sheets



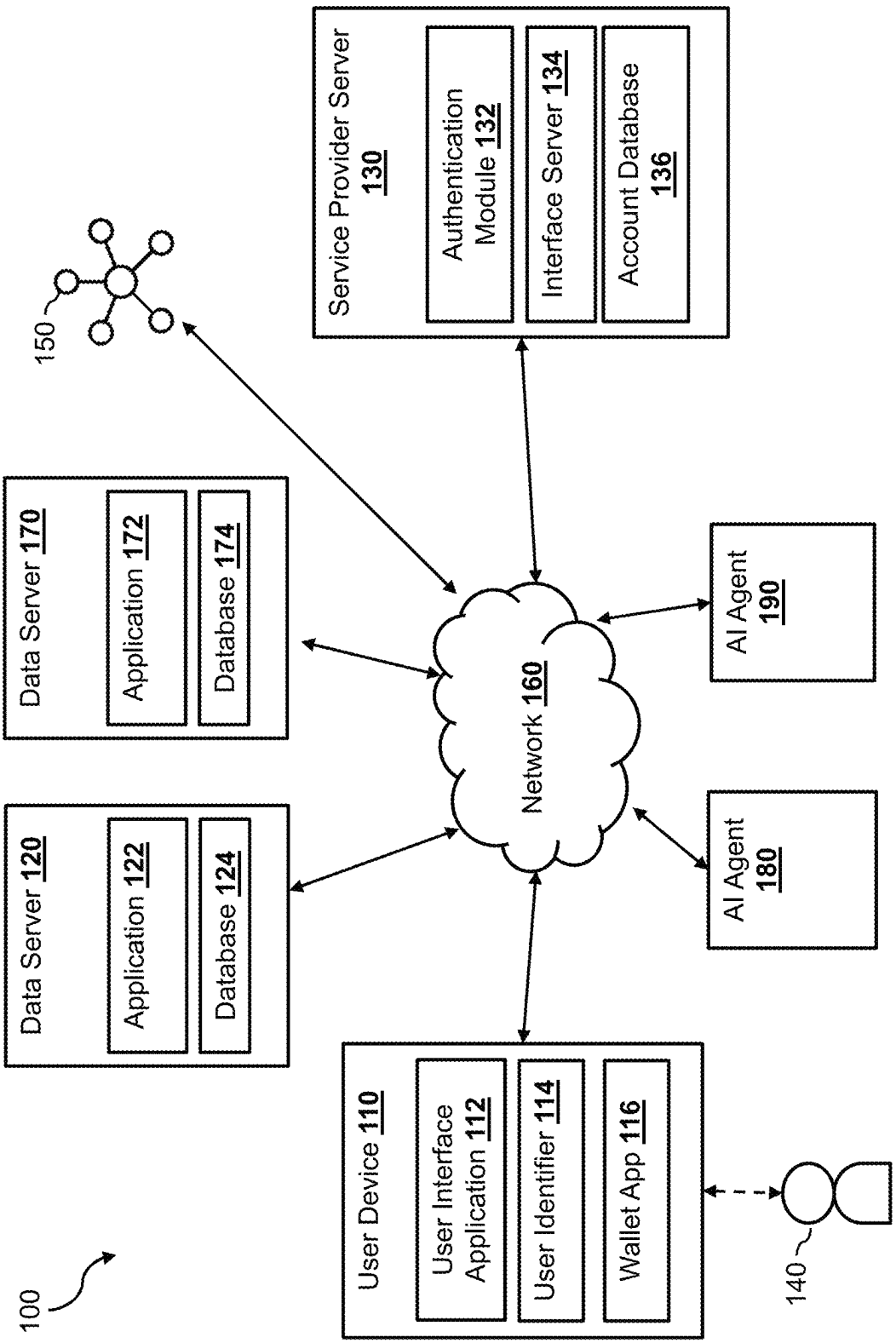

100
User Device 110
User Interface Application 112
User Identifier 114
Wallet App 116
140
Data Server 120
Application 122
Database 124
Data Server 170
Application 172
Database 174
150
Service Provider Server 130
Authentication Module 132
Interface Server 134
Account Database 136
Network 160
AI Agent 180
AI Agent 190

Figure 1

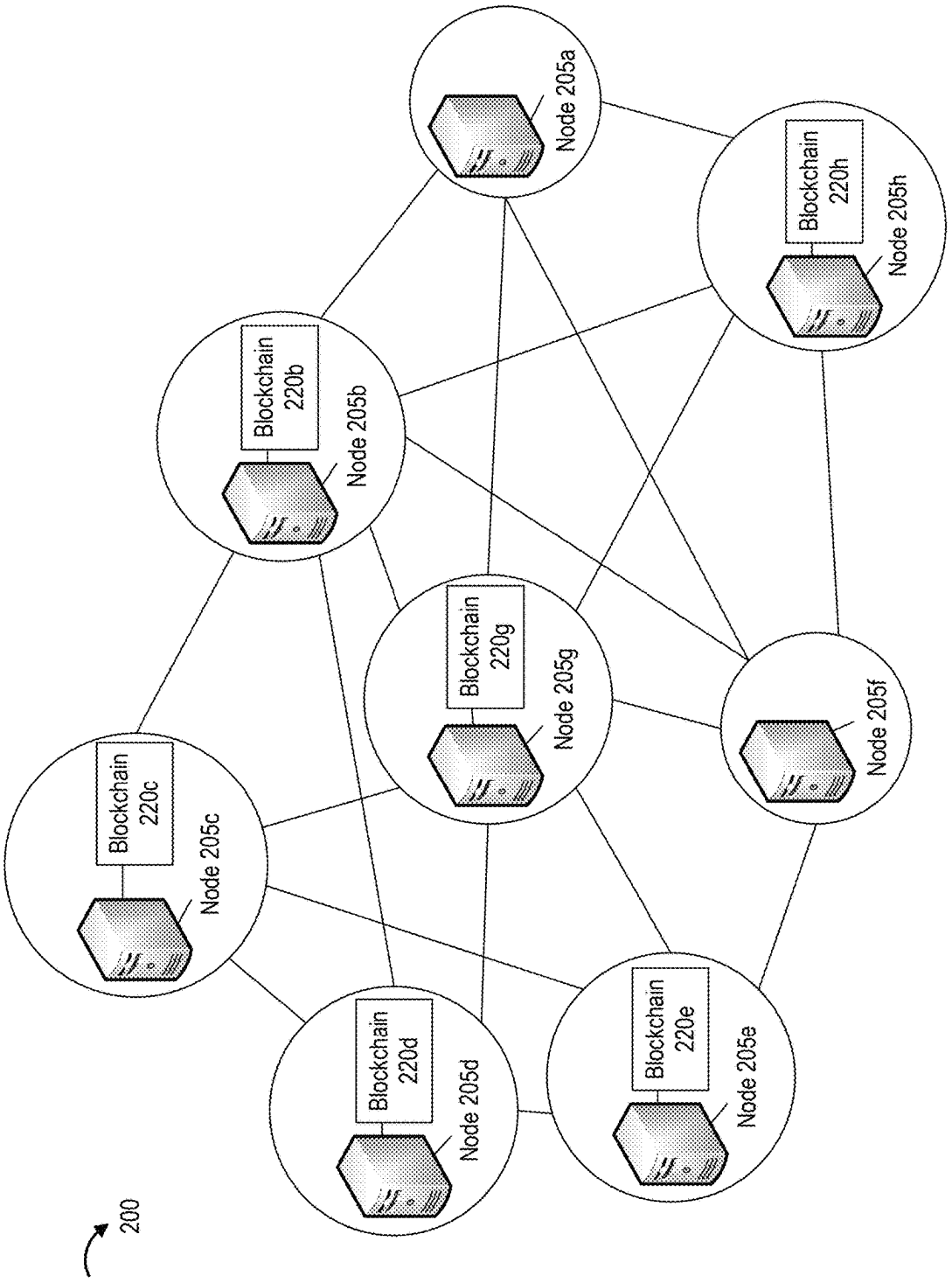

200
Node 205a
Blockchain 220h
Node 205h
Blockchain 220b
Node 205b
Blockchain 220g
Node 205g
Node 205f
Blockchain 220c
Node 205c
Blockchain 220d
Node 205d
Blockchain 220e
Node 205e

FIG. 2

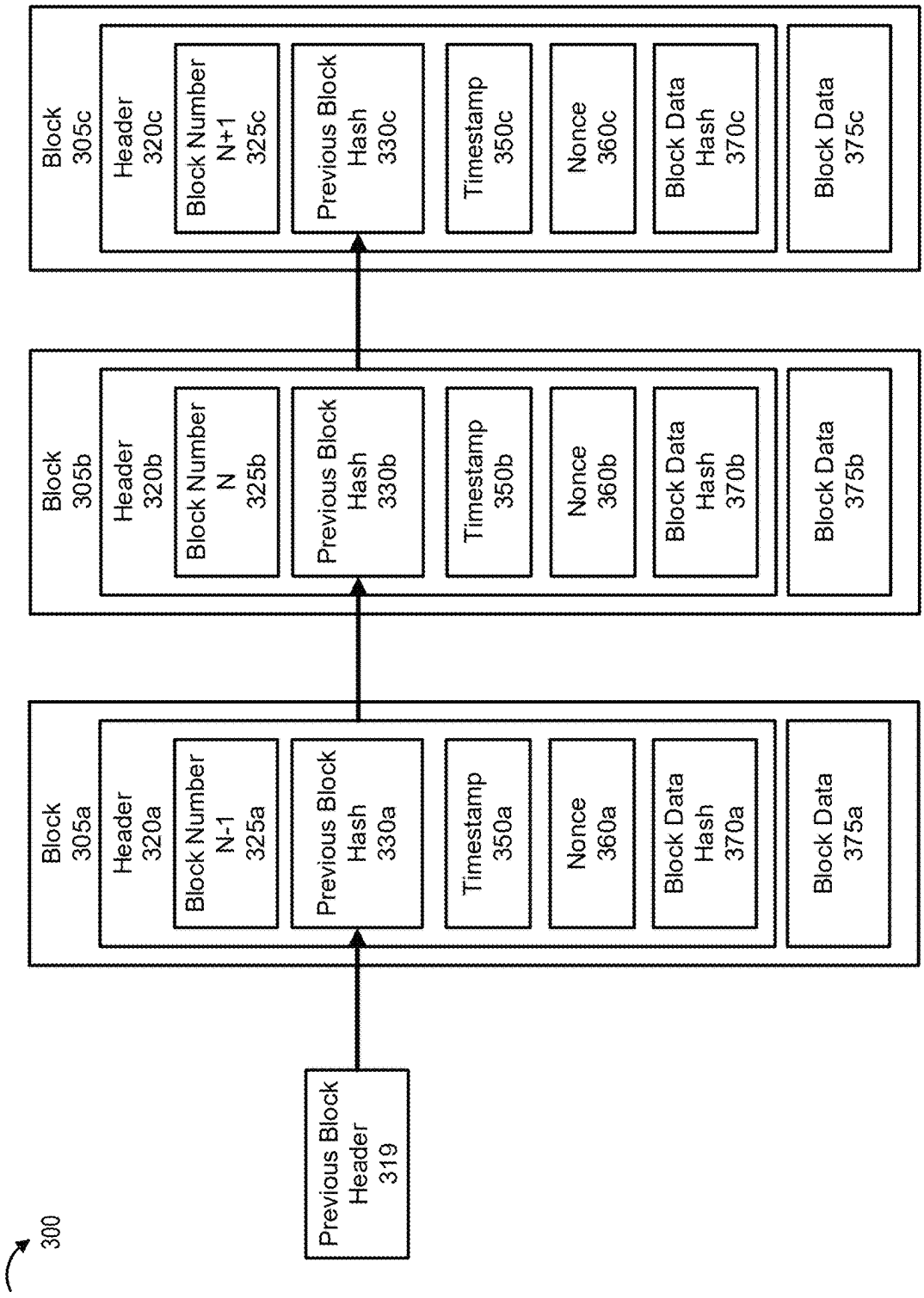

300
Previous Block Header 319
Block 305a
Header 320a
Block Number N-1 325a
Previous Block Hash 330a
Timestamp 350a
Nonce 360a
Block Data Hash 370a
Block Data 375a
Block 305b
Header 320b
Block Number N 325b
Previous Block Hash 330b
Timestamp 350b
Nonce 360b
Block Data Hash 370b
Block Data 375b
Block 305c
Header 320c
Block Number N+1 325c
Previous Block Hash 330c
Timestamp 350c
Nonce 360c
Block Data Hash 370c
Block Data 375c

FIG. 3

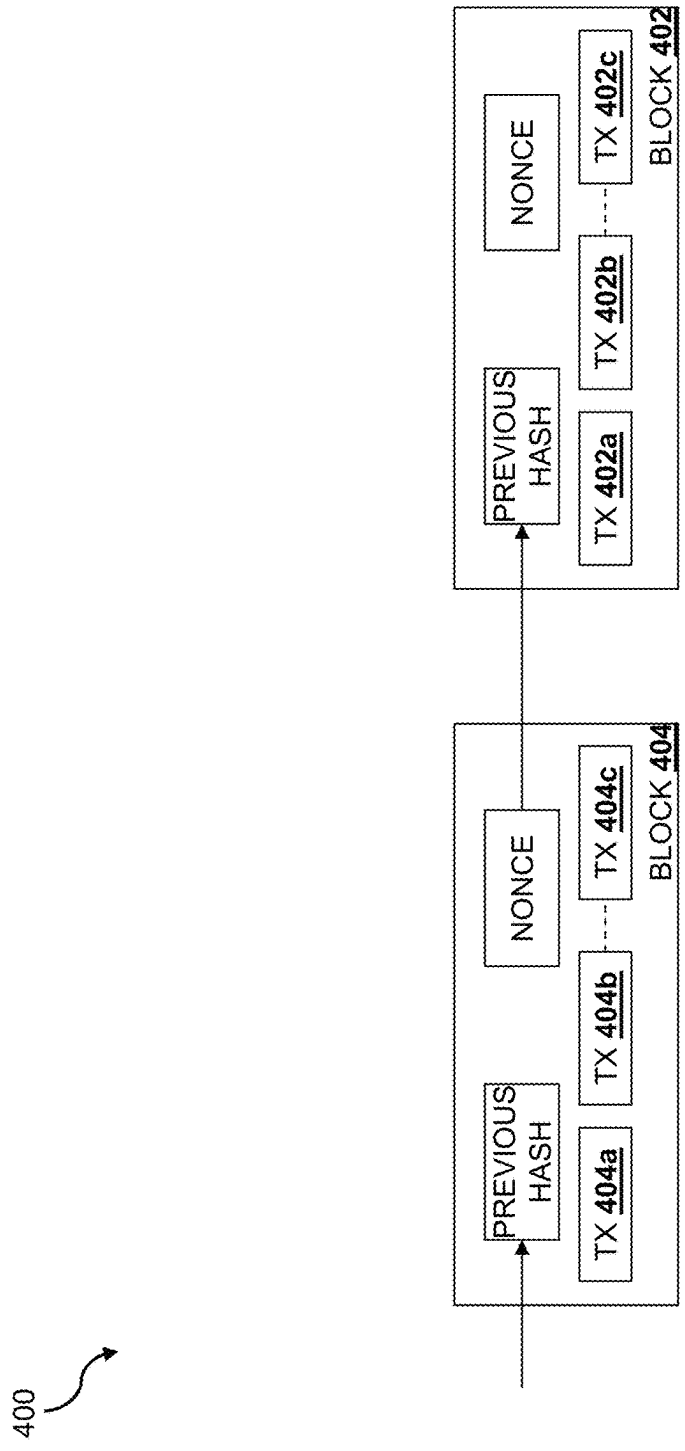

400
BLOCK 404
PREVIOUS HASH
NONCE
TX 404a
TX 404b
TX 404c
BLOCK 402
PREVIOUS HASH
NONCE
TX 402a
TX 402b
TX 402c

Figure 4

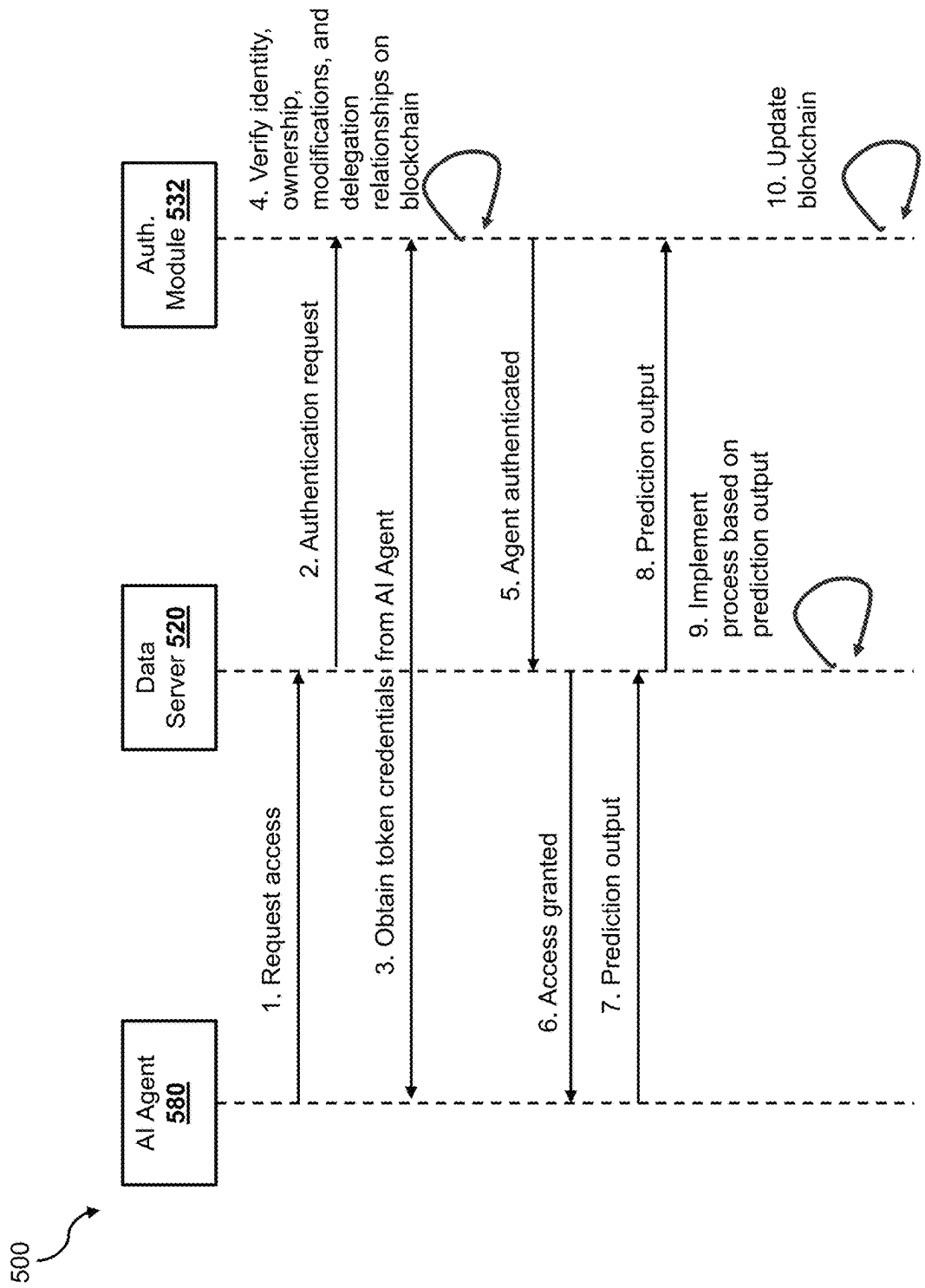

500
AI Agent 580
Data Server 520
Auth. Module 532
1. Request access
2. Authentication request
3. Obtain token credentials from AI Agent
4. Verify identity, ownership, modifications, and delegation relationships on blockchain
5. Agent authenticated
6. Access granted
7. Prediction output
8. Prediction output
9. Implement process based on prediction output
10. Update blockchain

Figure 5

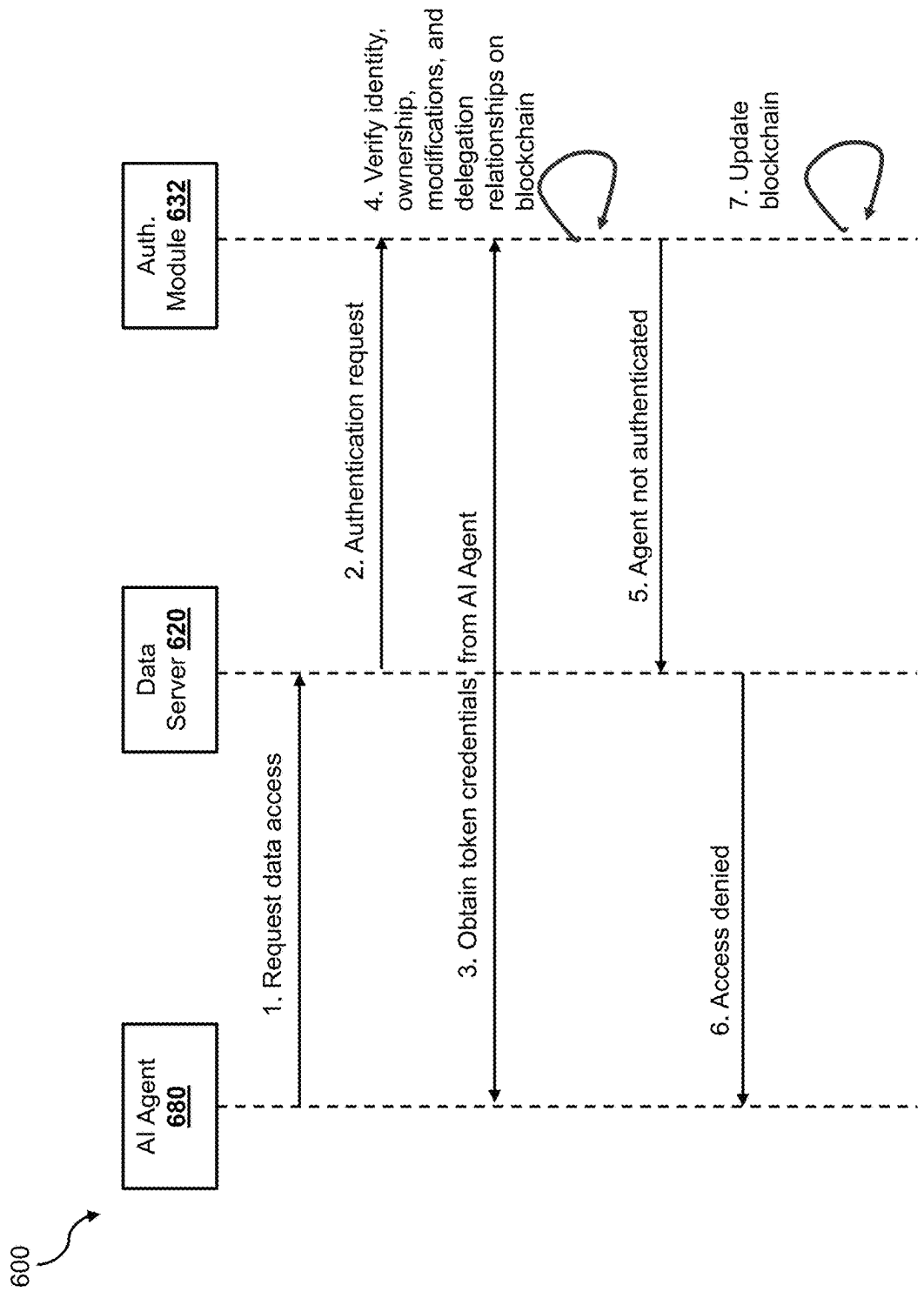

600
AI Agent 680
Data Server 620
Auth. Module 632
1. Request data access
2. Authentication request
3. Obtain token credentials from AI Agent
4. Verify identity, ownership, modifications, and delegation relationships on blockchain
5. Agent not authenticated
6. Access denied
7. Update blockchain

Figure 6

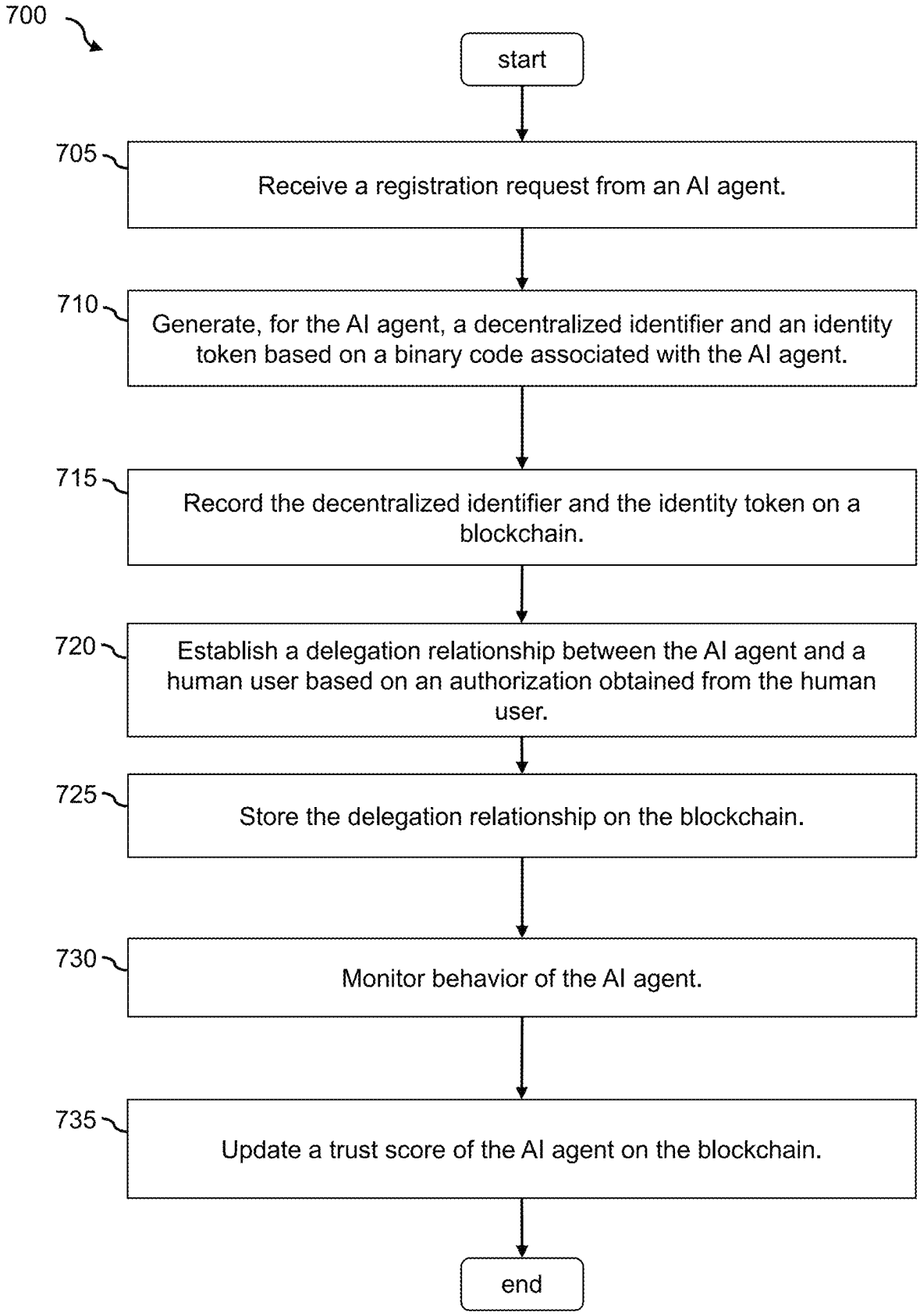

700
start
705
Receive a registration request from an AI agent.
710
Generate, for the AI agent, a decentralized identifier and an identity token based on a binary code associated with the AI agent.
715
Record the decentralized identifier and the identity token on a blockchain.
720
Establish a delegation relationship between the AI agent and a human user based on an authorization obtained from the human user.
725
Store the delegation relationship on the blockchain.
730
Monitor behavior of the AI agent.
735
Update a trust score of the AI agent on the blockchain.
end

Figure 7

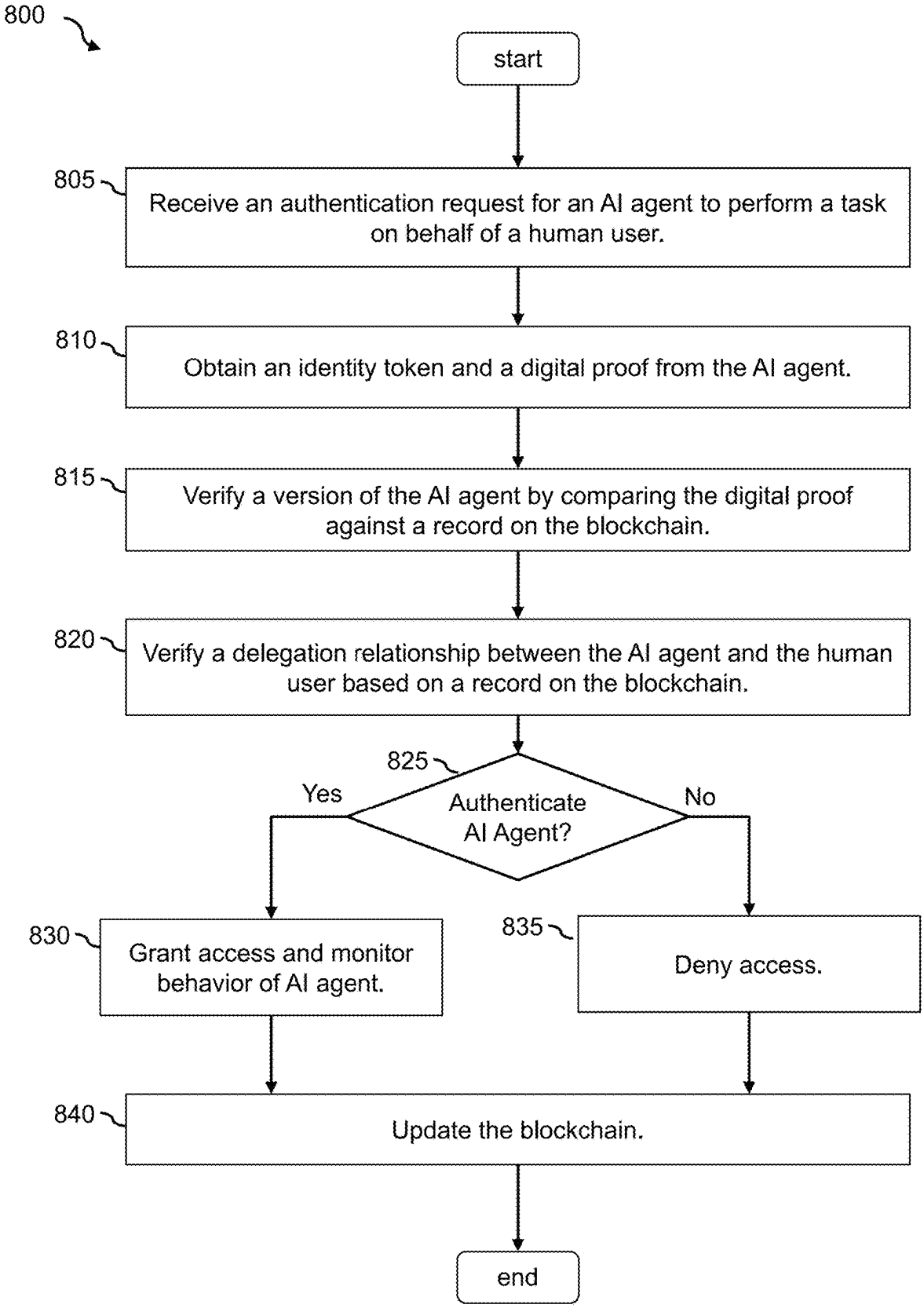

800
start
805
Receive an authentication request for an AI agent to perform a task on behalf of a human user.
810
Obtain an identity token and a digital proof from the AI agent.
815
Verify a version of the AI agent by comparing the digital proof against a record on the blockchain.
820
Verify a delegation relationship between the AI agent and the human user based on a record on the blockchain.
825
Authenticate AI Agent?
Yes
No
830
Grant access and monitor behavior of AI agent.
835
Deny access.
840
Update the blockchain.
end

Figure 8

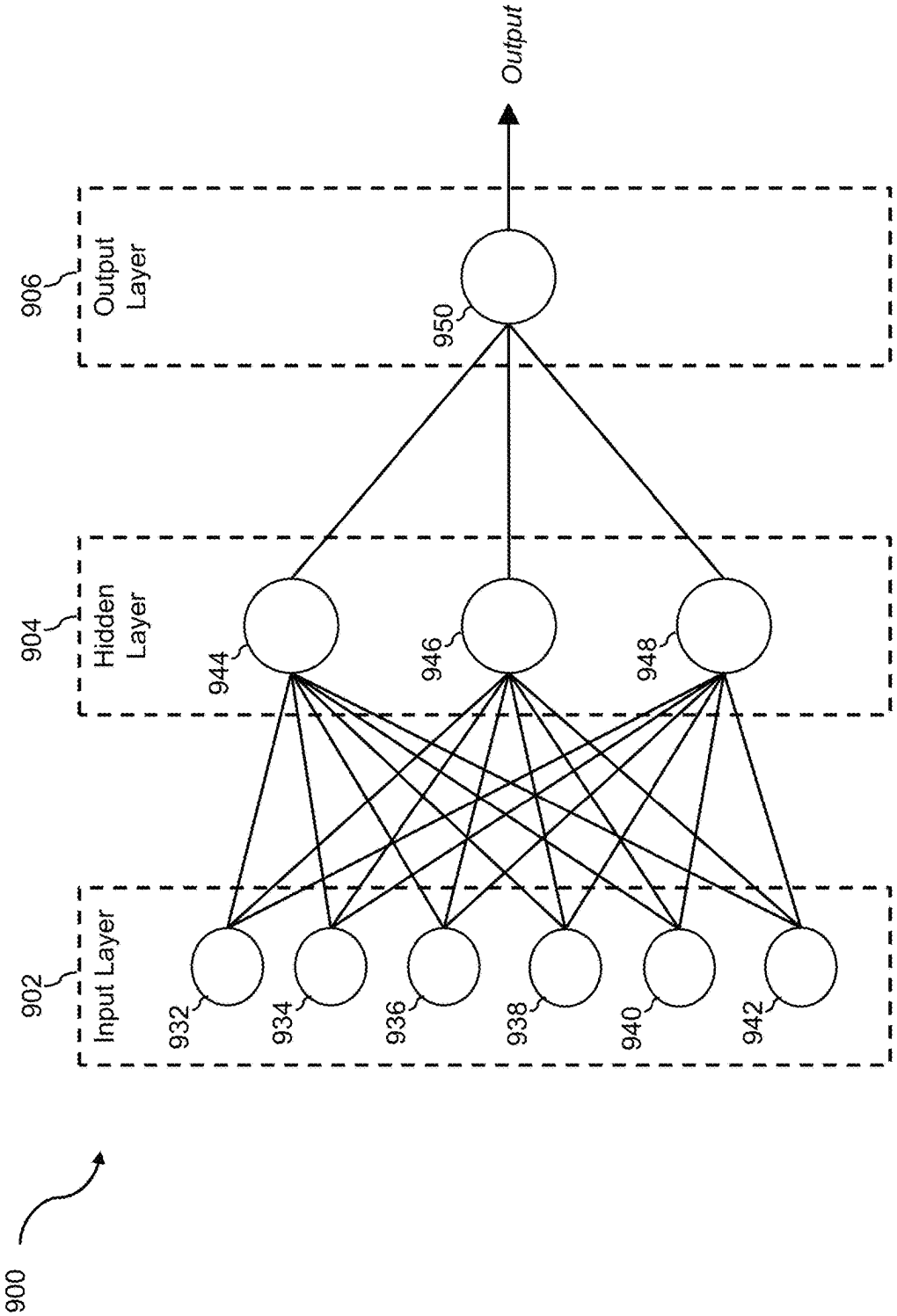

Output
906
Output Layer
950
904
Hidden Layer
944
946
948
902
Input Layer
932
934
936
938
940
942
900

Figure 9

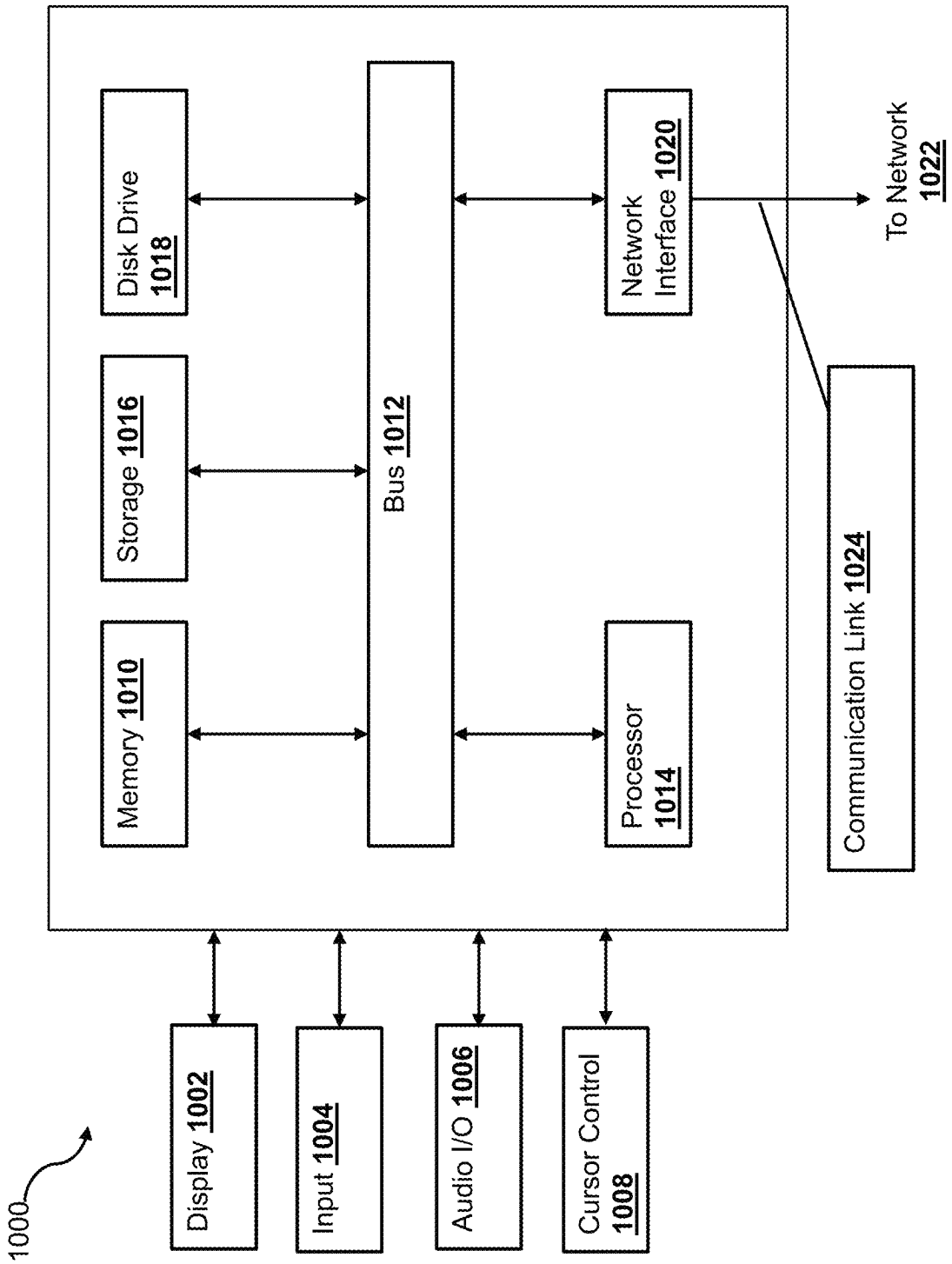

1000
Display 1002
Input 1004
Audio I/O 1006
Cursor Control 1008
Memory 1010
Storage 1016
Disk Drive 1018
Bus 1012
Processor 1014
Network Interface 1020
To Network 1022
Communication Link 1024

Figure 10

BLOCKCHAIN-BASED ARTIFICIAL INTELLIGENCE AGENT LIFE CYCLE MANAGEMENT AND AUTHENTICATION SYSTEMS AND METHODS

BACKGROUND

The present specification generally relates to computer network security, and more specifically, to providing a blockchain-based framework for facilitating authentication and life-cycle management for artificial intelligence agents according to various embodiments of the disclosure.

RELATED ART

Artificial intelligence agents, commonly known as AI agents or virtual assistants, can be applied to a wide range of practical applications across various industries. In customer service, AI agents can handle user inquiries, provide support, and resolve issues 24/7, improving customer satisfaction and reducing operational costs. In healthcare, AI agents can offer initial consultations, answer health-related questions, and remind patients to take their medications. In the e-commerce sector, AI agents can assist with product recommendations, order tracking, and personalized shopping experiences. In information technology (IT) support, these agents can guide users through troubleshooting steps, helping them resolve software and hardware issues. Specifically, for network hazards, AI agents can diagnose connectivity problems, suggest corrective actions, and provide step-by-step guidance to ensure network security and stability. Their versatility and ability to handle diverse tasks make them valuable tools in enhancing efficiency and user experience in various fields.

AI agents often employ a neural network based generative language model (also referred to as "AI models") to generate an output such as in the form of a text response, or a series actions to complete a complex task, such as to network issue troubleshooting, etc. Such generative language model receives a natural language input in the form of a sequence of tokens, and in turn generates a predicted distribution over a token space conditioned on the input sequence. Generated output tokens over time may in turn form the text response, or actions for completing the task.

Due to the capabilities of AI agents, they are often utilized to perform tasks autonomously or on behalf of users. For example, an AI agent may be instructed by a user to perform payment transactions on behalf of the user using an account of the user. In another example, an AI agent may be instructed by a user to analyze a computer network issue, and implement a plan to resolve the computer network issue. While these AI agents offer tremendous functionalities and convenience for human users, they also bring forth potential computer security risks to computer networks of various organizations. For example, in order for the AI agent to perform the tasks, it may be required that the AI agent access certain resources (e.g., data, functionalities of another computer module, such as another AI agent, etc.). Without proper control and management of these AI agents, they may be utilized by malicious users to gain unauthorized access to these resources. As such, there is a need to provide a framework for managing the life cycle and authentication of AI agents.

BRIEF DESCRIPTION OF THE FIGURES

FIG. **1** is a block diagram illustrating a networked system that includes an electronic transaction system according to an embodiment of the present disclosure;

FIG. **2** illustrates an example blockchain network according to an embodiment of the present disclosure;

FIG. **3** illustrates an example blockchain according to an embodiment of the present disclosure;

FIG. **4** is a schematic view illustrating a token transaction ledger according to an embodiment of the present disclosure;

FIG. **5** is swim lane diagram illustrating example interactions among multiple devices for authenticating an AI agent e according to an embodiment of the present disclosure;

FIG. **6** is swim lane diagram illustrating example interactions among multiple devices for authenticating an AI agent e according to an embodiment of the present disclosure;

FIG. **7** illustrates a process of registering an AI agent according to an embodiment of the present disclosure; and

FIG. **8** illustrates a process of authenticating an AI agent according to an embodiment of the present disclosure;

FIG. **9** illustrates an example neural network that can be used to implement a machine learning model according to an embodiment of the present disclosure; and

FIG. **10** is a block diagram of a system for implementing a device according to an embodiment of the present disclosure.

Embodiments of the present disclosure and their advantages are best understood by referring to the detailed description that follows. It should be appreciated that like reference numerals are used to identify like elements illustrated in one or more of the figures, wherein showings therein are for purposes of illustrating embodiments of the present disclosure and not for purposes of limiting the same.

DETAILED DESCRIPTION

The present disclosure includes methods and systems for providing a framework that manages the life cycle and authentication of artificial intelligence (AI) agents in various computer network environments. As discussed herein, AI agents can be widely deployed in different computer environments and settings to perform different tasks either autonomously or on behalf of different human users (also referred to as "delegators"). An AI agent may access resources (e.g., data, functionalities associated with another computer module such as another AI agent, etc.) within a computer network (e.g., within an internal private network associated with an organization, within a public network, etc.), perform various processing based on the resources, and generate outputs (e.g., a prediction of a computer network issues, a prediction of a medical diagnosis, etc.). The outputs may be used by the AI agent, a human user that controls the AI agent, and/or another computer module (e.g., another AI agent) for other processes (e.g., implementing a fix for the computer network issue, etc.).

While these AI agents offer tremendous capabilities and convenience for human users, their frequent access to various computer resources can pose significant risks to the computer network where resources are accessed. For example, an AI agent may be developed and used by a malicious delegator (e.g., a malicious human user, etc.) to perform unauthorized tasks within a computer network environment. In another example, an AI agent may be modified by a malicious party different from the delegator (e.g., infected with a computer virus, etc.), such that the AI agent may perform operations apart from the tasks delegated y the delegator (e.g., accessing data, and transmitting the data to an unauthorized external server, etc.).

As such, the framework provides techniques that utilize blockchains to manage the life cycle and authentication of the AI agents, such that any AI agents requesting access to resources would need to be authenticated based on a blockchain-based authentication process before access is granted (or denied) to the AI agents. Under the framework, an authentication system uses a blockchain to track a life cycle for each AI agent (e.g., a model, a version, one or more updates to the version based on modifications to the binary code of the AI agent, tasks and actions performed by the AI agent, etc.), and use the information stored on the blockchain to verify various attributes of the AI agent (e.g., whether the AI agent satisfies a set of requirements for performing the requested task, whether the AI agent has been modified since the last time a record associated with the AI agent has been recorded on the blockchain, delegation relationships associated with the AI agent, etc.) before access to a resource is granted to the AI agent. This way, any action performed by the AI agent and/or action performed to the AI agent is monitored and recorded on the blockchain. Due to the immutability of blockchains, using such a technique to track a life cycle (and other activities) of AI agents and authenticate AI agents based on attributes stored on the blockchain can substantially enhance the computer network security where the AI agents are deployed.

FIG. **1** illustrates a networked system **100**, within which the framework may be implemented according to one embodiment of the disclosure. Note that the present techniques may be applied in many different computing and technological environments, however, and are not limited to those shown in the figures. The networked system **100** includes a service provider server **130**, data servers **120** and **170**, a user device **110** associated with a user **140**, and AI agents **180** and **190** that may be communicatively coupled with each other via a network **160**. The network **160**, in one embodiment, may be implemented as a single network or a combination of multiple networks. For example, in various embodiments, the network **160** may include the Internet and/or one or more intranets, landline networks, wireless networks, and/or other appropriate types of communication networks. In another example, the network **160** may comprise a wireless telecommunications network (e.g., cellular phone network) adapted to communicate with other communication networks, such as the Internet.

Each of the AI agents **180** and **190** may be implemented as a neural network-based generative language model, such as a large language model (LLM). An LLM is designed to understand and generate human languages. An LLM may adopt a Transformer architecture that often entails a significant number of parameters (neural network weights) and computational complexity. For example, LLM such as Generative Pre-trained Transformer (GPT) 3 has 175 billion parameters, Text-to-Text Transfer Transformers (T5) has around 11 billion parameters. An LLM may comprise an architecture of mixed software and/or hardware, e.g., including an application-specific integrated circuit (ASIC) such as a Tensor Processing Unit (TPU). Each of the AI agents **180** and **190** may be implemented within a device, such as the user device **110** or a computer server communicatively connected to the network **160**, such that it is accessible by the user device **110** and/or other devices.

The user device **110**, in one embodiment, may be utilized by the user **140** to interact with the data servers **120** and **170**, the AI agents **180** and **190**, and/or the service provider server **130** over the network **160**. For example, the user **140** may use the user device **110** to interact with (or use one of the AI agents **180** or **190** to interact with) one or more of the data servers **120** and/**170** (e.g., to register an account with the data servers **120** and **170**, to access data stored in the data server **120** and **170**, to use one or more functionalities provided by a computer module, such as applications **122** or **172** associated with the data servers **120** and **170**, etc.). Additionally, the user **140** may also interact with the service provider server **130** to register an account with the service provider server **130**. By registering an account with the service provider server **130**, the service provider server **130** of some embodiments may automatically use the credential information (e.g., biometric data, such as facial features, fingerprint, etc. associated with the user **140**, password associated with the user **140**, device identifier associated with the user **140**, etc.) to register the user with the service provider server **130**. The user **140** may also register one or more AI agents (e.g., the AI agents **180** and **190**, etc.) with the service provider server **130**.

Under the framework, either a human user and or an AI agent may be assigned a decentralized identity when the human user or the AI agent is registered with the service provider server **130**, which includes an authentication module **132**, an interface server **134**, and an account database **136**. For example, when a human user or an AI agent requests to be registered within the framework, the authentication module **132** of the service provider server may generate an identifier (e.g., a randomly generated string or value, a token associated with a blockchain **150**, etc.). The identifier may be stored on the blockchain **150** and associated with the human user or the AI agent. When the identifier is generated for a human user, the identifier may be associated through the life of the human user. When the identifier is generated for an AI agent, the identifier may be associated with the AI agent through the life cycle (e.g., from development and deployment, different versions and upgrades, transferring ownership between different users (delegators), until the AI agent is retired or destroyed, etc.).

In addition to the identifiers, the authentication module **132** may also generate an identity token (e.g., minting a token for the blockchain **150**, etc.) for the human user or the AI agent. For a human user, the authentication module **132** may link the identity token to the identifier of the user **140** and the credentials (e.g., a biometric signature, multi-factor authentication data, which may include a passcode and a device identifier of a device associated with the user **140**, such as the user device **110**, etc.). The identity token generated for the user may be stored in a digital wallet of the user **140**. The authentication module **132** may store, on the blockchain **150**, a record that links the identity token to the user **140** (e.g., the credential information, etc.) and the digital wallet.

For an AI agent, the authentication module **132** may link the identity token to the identifier of the AI agent. The authentication module **132** may also store, on the blockchain **150**, a record that links the identity token of the AI agent to various attributes associated with the AI agent, such as an AI agent type (e.g., an autonomous agent, a supervised agent, etc.), task permissions associated with the AI agent (e.g., access to computer network data, access to medical data, access to financial transaction data, etc.), delegation relationships when the AI agent is a supervised agent (e.g., one or more human users who can delegate task to the AI agent, etc.), a current model version of the AI agent, and instructions for destroying or updating the AI agent when the AI agent is retired. The identity token may be provided to the AI agent, such that the AI agent can use/present it during subsequent authentication processes. In some embodiments, the authentication module **132** may also obtain a hash value

of a binary code of the AI agent (or binary code of other AI agent of the same model and version), and link the hash value to the identity token issued for the AI agent.

The hash value may be generated for an AI agent based on different factors, such as model weights (e.g., parameters associated with the AI agent, etc.), a neural network architecture of the AI agent (e.g., the layer structures, the hyperparameters, etc.), training dataset signature (e.g., training data used to train the AI model), the source code of the AI agent, owner/delegator metadata (e.g., public key signatures of identity metadata of the owner/delegator of the AI agent that is stored on the blockchain **150**). In some embodiments, the authentication module **132** may generate a hash value for each of the factors, and then combine them (e.g., append to each other) to generate combined hash, which can be used as a persistent fingerprint of the AI agent in its current configuration.

Whenever attributes of the AI is changed (e.g., upgraded to a new model version, any modification to the binary code, a change of ownership, a change of a delegator, undergo a re-training, re-deployed at another machine, etc.), the AI agent has to re-register itself with the authentication module **132**, such that a new identity token (which is linked to the updated version/binary code of the AI agent) can be issued to the AI agent. The linked information for a human user or an AI agent can be stored on the blockchain **150** or a data store that is linked to the blockchain **150**.

The user **140** may also interact with the AI agents **180** and **190**. For example, after registering an AI agent (e.g., the AI agent **180**, the AI agent **190**, etc.) with the authentication module **132**, and specify a delegation relationship with the AI agent, the user **140** may begin using the AI agent to perform various tasks on behalf of the user **140**. For example, if the user **140** is an information technology (IT) specialist, the user **140** may use the AI agent to analyze a computer network condition and determine an issue with the computer network. The user **140** may also use the AI agent to generate a solution to fix the computer network issue. Based on the output from the AI agent, the user **140** (or the AI agent automatically) may interact with other computer components within the computer network to implement the fix. In another example, if the user **140** is a medical professional, the user **140** may use the AI agent to analyze heath data (e.g., an X-ray, markers from a blood test, etc.) of a patient and determine a diagnosis of the patient. The user **140** may also use the AI agent to generate a treatment or a prescription for the user **140** based on the diagnosis. When performing any one of these tasks, the AI agent may need to interact with the data server **120** and/or the data server **170** (e.g., to access resources from the data server **120** and/or the data server **170**, such as data or functionalities provided by one or more software modules of the data server **120** and/or the data server **170**). As such, each of the AI agents **180** and **190** may also interact with the data servers **120** and **170** either autonomously or on behalf of a delegator.

The user device **110**, in various embodiments, may be implemented using any appropriate combination of hardware and/or software configured for wired and/or wireless communication over the network **160**. In various implementations, the user device **110** may include at least one of a wireless cellular phone, wearable computing device, PC, laptop, etc.

The user device **110**, in one embodiment, includes a user interface (UI) application **112** (e.g., a web browser, a mobile application, etc.), which may be utilized by the user **140** to interact with the data servers **120** and **170**, the AI agents **180** and **190**, and/or the service provider server **130** over the network **160**. In one implementation, the UI application **112** includes a software program (e.g., a mobile application) that provides a graphical user interface (GUI) for the user **140**. In another implementation, the UI application **112** includes a browser module that provides a network interface to browse information available over the network **160**. For example, the UI application **112** may be implemented, in part, as a web browser to view information available over the network **160**.

The user device **110**, in one embodiment, may include at least one identifier **114**, which may be implemented, for example, as operating system registry entries, cookies associated with the UI application **112** and/or the wallet application **116**, identifiers associated with hardware of the user device **110** (e.g., a media control access (MAC) address), or various other appropriate identifiers. In various implementations, the identifier **114** may be passed with a user login request to the service provider server **130** via the network **160**, and the identifier **114** may be used by the service provider server **130** to associate the user **140** with a particular user account, a particular digital wallet, and/or a particular profile.

The user device **110** also includes a wallet application **116** configured to store data associated with the user (e.g., the identifier token that was issued for the user **140**, the identifier and identifier token associated with one or more AI agents, credit cards, debit cards, gift cards, a driver's license, a passport, etc.) and to communicate with other devices (e.g., the authentication module **132**, the AI agents **180** and **190**, the data servers **120** and **170**, etc.) to perform various tasks for the user **140**.

In various implementations, the user **140** is able to input data and information into an input component (e.g., a keyboard) of the user device **110**. For example, the user **140** may use the input component to interact with the UI application **112** (e.g., to retrieve data from third-party servers such as the data servers **120** and **170**, to provide instructions, in the form of a prompt, to the AI agents **180** and **190** to instruct the AI agents to perform tasks on behalf of the user **140**, etc.).

While only one user device **110** is shown in FIG. **1**, it has been contemplated that multiple user devices can be included in the networked system **100**. Each of the user devices may include similar hardware and software components as the user device **110** to enable their respective users to interact with the data servers **120** and **10**, the AI agents **180** and **190**, and/or the service provider server **130**.

Each of the data servers **120** and **170** may be associated with a third-party organization, such as a company, a hospital, a medical facility, a government agency that is configured to store data associated with various users and entities. As shown, each of the data servers **120** and **170** may include a database (e.g., databases **124** and **174**, respectively) for storing data usable for performing various tasks, such as computer network diagnosis, medial diagnosis, etc. Each of the data servers **120** and **170** may also include one or more computer software modules (e.g., applications **122** and **172**, respectively) for performing various functionalities. Each of the applications **122** and **172** may be implemented as a computer program, such as another AI agent, a web application, a database application, etc. The functionalities provided by the application **112** and **172** may include

In some embodiments, the user **140**, via the UI application **112**, may access the data stored in the databases **124** and **174** and/or utilize the functionalities provided by the applications **112** and **172**. Additionally or alternatively, the user **140** may instruct the AI agents **180** and **190** to access the data and/or

the functionalities of the data servers **120** and **170** on behalf of the user **140**. For example, the user **140** may instruct (e.g., through one or more prompts) an AI agent to perform a task (e.g., diagnosing a computer network issue of a computer network, determining a medical condition of a patient, etc.). In order to perform the task, the AI agent may need data (e.g., computer network data, patient health data, etc.) from the databases **124** and/or **174**. As such, while performing the task, the AI agent may request access to data stored in the databases **124** and/or **174**, or the access to the functionalities provided by the applications **112** and/or **172**.

When the AI agent requests access to any resources from the data server **120** and/or the data server **170**, a blockchain-based authentication process may be triggered. In some embodiments, at least part of the blockchain-based authentication process may be implemented as a computer process executed by the authentication module **132** and/or a smart contract associated with the blockchain **150**. For example, the authentication module **132** may generate a smart contract that includes executable code associated with the blockchain-based authentication process, and store it in the blockchain **150**. When a data server (e.g., the data server **120** and/or the data server **170**, etc.) receives a request to access a resource from an AI agent, the data server may access the smart contract on the blockchain **150**, and trigger an execution of the smart contract **150**. Alternatively, the data server may transmit an authentication request to the authentication module **132**, and the authentication module **132** may initiate the blockchain-based authentication process.

The blockchain-based authentication process may begin with requesting the AI agent to provide an identity token and a proof of state. The identity token may be the token issued to the AI agent when the AI agent registered with the authentication module **132**. The proof of state may include information that proves the state of the binary code associated with the AI agent. For example, the proof of state may include a hash of the binary code of the AI agent, generated at the time of the authentication process (e.g., computed by the smart contract or the authentication module **132**).

The authentication module **132** or the system executing the smart contract may access a record on the blockchain **150** using the identity token provided by the AI agent. The authentication module **132** or the system executing the smart contract may extract the information associated with the AI agent from the record, and compare the information against the proof of state provided by the AI agent. The comparison enables the authentication module **132** or the system executing the smart contract to determine whether the AI agent has been modified (e.g., upgraded, code modified by an entity, etc.) since the last time the AI agent registered with the authentication module **132**. Since the AI agent is required to re-register whenever any changes to the code of the AI agent is made, a determination that the AI agent has been modified since the last time it registered with the authentication module **132** may indicate that the AI agent has been compromised (e.g., taken over by a malicious user, infected by a code, such as a virus, etc.). As such, the authentication module **132** or the system executing the smart contract may deny the request to access the resource based on failing to authenticate the AI agent.

On the other hand, if the authentication module **132** or the system executing the smart contract determines that the AI agent has not been modified since the last time it registered with the authentication module **132** based on the comparison, the authentication module **132** or the system executing the smart contract may continue the authentication process for the AI agent. For example, the authentication module **132** or the system executing the smart contract may determine whether the AI agent has permissions to access the resource indicated in the request. The authentication module **132** or the system executing the smart contract may extract the permission information from one or more records from the blockchain based on the identity token provided by the AI agent, and compare the permission information against the resource. The authentication module **132** or the system executing the smart contract may deny access to the resource if it determines that the AI agent has no permission to access the resource based on the permission information.

If it is determined that the AI agent is performing a task on behalf of a human user (e.g., based on the identity token and/or the request, etc.), the authentication module **132** or the system executing the smart contract may request the AI agent to provide a token associated with the human user. For example, when the human user instructs the AI agent to perform a task, the human user may provide the identity token of the human user to the AI agent as part of the prompt (which the AI agent may provide to the authentication module **132** during the authentication process). The authentication module **132** or the system executing the smart contract may access the delegation relationship associated with the AI agent from the information stored on the blockchain **150** based on the token associated with the AI agent. The authentication module **132** or the system executing the smart contract then determines if the human user who delegated the task to the AI agent has an established (e.g., registered) delegation relationship with the AI agent according to the blockchain **150** (e.g., has the relationship been recorded on the blockchain **150**). If the human user who delegated the task to the AI agent does not have a registered delegation relationship with the AI agent, the authentication module **132** or the system executing the smart contract may deny the request to access the resource.

On the other hand, if the human user who delegated the task to the AI agent has a registered delegation relationships with the AI agent, the authentication module **132** or the system executing the smart contract may determine whether the task is similar to other tasks that the AI agent has performed on behalf of that human user in the past. For example, since every task (e.g., resource access operation) performed by the AI agent has been recorded on the blockchain **150**, the authentication module **132** or the system executing the smart contract may retrieve the information associated with the tasks performed by the AI agent on behalf of the human user in the past from the blockchain **150**. The authentication module **132** or the system executing the smart contract may analyze the task information and determine whether the task associated with the request is consistent with the previous task (e.g., whether the task deviates in terms of the type of resources being accessed from the previous tasks). For example, if the previous tasks involve the AI agent accessing funding account information of the human user, the authentication module **132** or the system executing the smart contract may determine that the current task is consistent with the previous tasks if the current task requires access to funding account information of the human user. But the authentication module **132** or the system executing the smart contract may determine that the current task is inconsistent with the previous tasks if the current task requires access to medical information of the human user. In some embodiments, the authentication module **132** or the system executing the smart contract may use a machine learning model to determine a deviation score between the current task and the previous tasks based on

attributes of the tasks, and determine to deny the request if the deviation score exceeds a threshold.

In some embodiments, the authentication module **132** or the system executing the smart contract may also determine a trust score for the AI agent based on the tasks previously performed by the AI agent and resources previously requested by the AI agent. For example, all AI agents may be assigned a trust score when the AI agents register with the authentication module **132**. The trust score may be stored on the blockchain **150** as a transaction record associated with the corresponding AI agent. Each time the AI agent requests access for a resource, and was granted access to the resource, the authentication module **132** may increase the trust score for the AI agent, and record the updated trust score on the blockchain **150** as a new transaction record. On the other hand, when the AI agent requests resources that deviate from the resources previously accessed by the AI agent, the authentication module **132** may reduce the trust score, and record the updated trust score on the blockchain **150** as a new transaction record. As such, the authentication module **132** or the system executing the smart contract may access the current trust score associated with the AI agent. The authentication module **132** or the system executing the smart contract may deny the request if the trust score is below a threshold.

After granting or denying the request, the authentication module **132** or the system executing the smart contract may create another transaction record associated with the request. The transaction record may indicate the identity of the AI agent, the type of resources being requested, the identity of the delegator (if any), whether the request was granted or denied, and/or other information related to the request. The transaction record may then be stored on the blockchain **150** for use subsequently in authenticating the AI agent.

The authentication module **132** or the system executing the smart contract may also update the trust score associated with the AI agent based on whether the request was granted (e.g., the trust score would be increased) or denied (e.g., the trust score would be denied). The authentication module **132** or the system executing the smart contract may also store the updated trust score on the blockchain **150** as a new transaction record.

As discussed herein, the framework disclosed herein may give rise to different types of use cases. While several example use cases are described herein, the application of the framework can extend beyond the example use cases without departing from the spirit of this disclosure. The following use cases illustrate dynamic and continuous authentication of AI agents while they execute tasks. These examples showcase how the framework ensures security, trust, and real-time access control while preventing misuse.

Use Case 1: AI Agent Managing Financial Transactions (Banking & Payments)

Scenario: A personal AI financial assistant (e.g., an AI-powered robo-advisor) manages a user's bank account, paying bills, transferring money, and making investment decisions.

How Dynamic AI Authentication Works:

Step 1: AI Agent is Tokenized & Authenticated Before Execution

When the user creates an AI financial agent, it is issued a unique AI Agent

Identity Token (AIT) with specific permissions (e.g., "Allowed to make payments up to $500 per transaction"). The AI agent is linked to the user's Human Identity Token (HIT) for accountability. The bank recognizes AI agents separately from human logins and requires additional verification for transactions.

Step 2: AI Agent Requests Authorization to Act on Behalf of the User

When the AI agent attempts to make a payment, it presents both its AIT and the delegation token from the user. The bank (or the service provider server **130**) checks if the AI agent's trust score (based on past transactions) is intact. The AI agent's behavioral signature is checked to ensure it is following usual transaction patterns (e.g., payments to known vendors).

The AI agent's behavior profile can be implemented as a behavioral signature. Since AI agent's behavior evolves post-deployment, a runtime-based identity based on execution patterns can be generated. The behavior profile may be based on the AI agent's API call patterns (e.g., logs of the AI's interactions such as "requests EHR data 5× daily," etc.), decision flow similarity-consistency in AI's logic path for the same inputs, and computational footprint-resource usage, execution timing, and query patterns.

Step 3: Continuous Monitoring During Execution

If the AI agent suddenly initiates an unusually large transfer (e.g., $5,000 instead of $500), or making API calls that are not usually made by the AI agent (as specified in the behavioral profile, the smart contract flags it as suspicious behavior. The AI agent is forced to re-authenticate dynamically using: (1) Zero-Knowledge Proofs (ZKP) to confirm it's still linked to the user and/or real-time human validation (e.g., user receives a biometric verification request). If authentication fails, the AI's access token is revoked immediately.

Step 4: Behavioral-Based Access Modification

If the AI Agent consistently shows safe behavior, its trust score improves (e.g., increases), and it can execute subsequent transactions without re-authenticating frequently based on the trust score. If the AI is flagged for repeated suspicious actions, it is automatically downgraded (e.g., reduces trust score), requiring frequent authentication. If it is determined that the AI is compromised (e.g., modified by an attacker) (determined based on the proof of state is different from the stored hash value indicating the state of the AI agent during the previous transaction), its token is revoked, and it must be reissued. This ensures that an AI agent's identity and integrity remain intact throughout its lifecycle. Below is an example of detailed mechanism to address this issue.

TABLE 1

Methods of checking AI Agent's integrity

Method	What It Checks?	How It Works?
Binary Hash Verification	Ensures the AI agent's executable code has not been altered.	The system computes a hash (SHA-256, Merkle Tree, etc.) of the AI agent's binaries and compares it with the registered AI Identity Token (AIT).

TABLE 1-continued

Methods of checking AI Agent's integrity

Method	What It Checks?	How It Works?
AI Model Hashing	Checks if the AI's trained model has been modified.	AI weights and parameters are hashed and stored on-chain. Each execution checks if the current AI model hash matches the last known hash.
Real-Time Behavioral Anomaly Detection	Detects if AI behavior deviates from expected patterns.	If AI starts making unauthorized decisions, it triggers an automated integrity re-check.
Attestation from a Secure AI Execution Environment (TEE/SGX)	Verifies that AI code is running in a secure environment.	AI agents execute inside a Trusted Execution Environment (TEE) such as Intel SGX, which can verify code integrity at runtime.

Initial AI Agent Registration:

When an AI agent is created, a cryptographic hash (e.g., SHA-256, SHA-3, or Blake2) of its binary and model parameters is generated. This hash is stored in the AI Identity Token (AIT). A corresponding signed certificate is issued to verify authenticity.

Pre-Execution Check:

Each time the AI agent is executed, the system (e.g., the authentication module **132**, the smart contract, etc.) computes a new hash of the AI agent's binaries and model. The authentication module **132** compares it against the AIT-stored hash. If there is a mismatch, the authentication module **132** flags the AI agent as modified and triggers re-authentication.

Continuous Runtime Integrity Checks:

If an AI agent modifies itself (e.g., through adversarial attacks, tampering, or retraining), its hash will change. If a change is detected, the AI's token (AIT) is revoked. A re-authentication process (admin review or cryptographic attestation) is triggered.

Why This Works? Immutable Hashes ensure that even the smallest code changes of the AI agent are immediately detected. AIT acts as a cryptographic fingerprint, ensuring the AI agent cannot execute if altered.

AI Model Integrity Tracking

AI models can be tampered with, leading to unintended or malicious actions. To prevent this, we store each AI agent's parameters as a cryptographic commitment (Merkle Root) on-chain. It requires each AI agent's execution to verify its model hash against the stored hash.

Implementation: The AI model's weights, hyperparameters, and training data fingerprints are hashed and stored on a blockchain. If an AI agent's model is retrained or altered, it must recompute its model hash, and submit a new AI Identity Token request to the authentication module **132**, which ensures that the AI models are not covertly altered post-deployment.

Behavioral Anomaly Detection for AI Modification

Even if the binary integrity is intact, an AI can still behave unpredictably. Thus, AI trust verification extends beyond code integrity to behavioral authentication.

How It Works:

Each AI agent has a behavior profile based on expected API calls (API calls that the AI agent has historically been using, or API calls that AI agents of the same type and/or functionalities have historically been using, etc.), typical input-output patterns (based on the AI Agent's previous behavior or behavior of other AI Agents of the same type), interaction frequency with other systems, etc. If an AI agent suddenly starts executing unauthorized actions, an anomaly score is computed (or reducing the AI agent's trust score). If anomaly score exceeds a threshold (or the trust score is below a threshold), the AIT of the AI agent is temporarily revoked. AI agent must then pass a re-authentication process before executing again, which prevents AI agents from being exploited via adversarial manipulation.

Trusted Execution Environments (TEE) for AI Security

For additional security, AI agents may be required to be executed within a Trusted Execution Environment (TEE) (which is a computer system environment within a computer device that includes a processor and memory that is isolated from other portions of the computer device), such as: Intel SGX (Secure Guard Extensions), Google Confidential Compute, AWS Nitro Enclaves. TEEs ensure that AI software executes in a secure, tamper-proof environment. If AI agent tries to self-modify or execute unapproved instructions, TEE blocks execution and invalidates the AI Identity Token (AIT). TEE prevents runtime modification of AI agent binaries and models.

AI Modification Response Mechanism

If an AI agent fails an integrity check, the system (e.g., the authentication module **132**) immediately revokes the AI Identity Token (AIT), logs the integrity failure to the blockchain **150**, and requires re-authentication & re-issuance of the token before the AI can execute tasks again. This ensures modified AI agents cannot execute without explicit re-authentication.

Use Case 2: AI Agent Accessing Patient Health Data & Ordering Medications (Healthcare & Telemedicine)

Scenario: A medical AI assistant is granted limited access to a patient's electronic health records (EHRs) to review test results, suggest potential diagnoses, and/or order prescription refills on behalf of a licensed doctor.

How Dynamic AI Authentication Works:

Step 1: AI Agent Registers with a Medical Institution

AI agent is issued a Medical AI Identity Token (AIT), linked to the doctor's HIT. AI is only allowed to access specific patient records based on assigned roles (e.g., "Can view but not modify medical history").

Step 2: AI Agent Requests Data Access

AI agent requests access to a patient's MRI scan results. The hospital's EHR system (which may include an authentication module similar to the authentication module **132** of the service provider server) verifies the AI's token, ensuring that the AIT of the AI agent is active, is linked to the assigned doctor (the delegator), and the task has not exceeded its access scope (e.g., no modification rights).

Step 3: Real-Time Behavioral Authentication

The AI agent's behavior is compared against expected workflows. The expected workflows may be based on roles

associated with the delegator, but may also be determined using a machine learning model based on past decisions, real-time behavioral monitoring, and blockchain-backed auditing. This ensures AI agent operates within safe, ethical, and predictable boundaries. The system (e.g., the authentication module **132**) determines expected workflows using a combination of role-based access control (RBAC), historical workflow analysis, and AI-specific behavioral modeling. Below is a breakdown of how this is achieved.

1. Role-Based Access Control (RBAC) for AI Agent Workflow Definitions

Since AI agents are designed to act within predefined roles (e.g., assisting doctors), expected workflows are directly tied to the roles and permissions assigned to both the AI agent and the human user (doctor, nurse, administrator, etc.).

How Expected Workflows Are Defined:

The doctor's role (e.g., the delegator, such as a surgeon, a general practitioner, etc.) determines which AI agent functions are authorized. The AI agent is issued a Delegated Task Token (DTT) that only permits actions within the scope of the doctor's role. The system rejects unauthorized actions outside the assigned workflow.

Example: A doctor's AI assistant can access medical imaging results but cannot modify EHR records without explicit approval. If the AI agent suddenly tries to prescribe medication without a doctor's authorization, it triggers an anomaly alert.

Workflow Definition Through Pre-Trained AI Models

Beyond role-based permissions, expected workflows are also defined using historical data and predefined medical protocols.

How It Works:

Pre-Trained AI Models are built using: clinical guidelines (e.g., WHO, FDA, HL7 FHIR standards), institutional protocols (hospital workflow data), previous doctor-AI interactions (machine learning on past decisions), etc.

Real-Time Workflow Matching:

Every request made by an AI agent is compared to historically validated workflows. If an action performed by the AI agent deviates significantly (e.g., exceeding a threshold), it is flagged as an anomaly. The system may transmit an alert to a device of the delegator or owner (e.g., the doctor or administrator, etc.) for approval.

Example: If an AI agent specialized in radiology normally flags suspicious scans for review, but suddenly tries to approve a diagnosis on its own, the system flags this as a deviation. The doctor must explicitly approve the AI agent's new behavior before it proceeds.

Behavioral Monitoring & AI Trust Score

Since AI models learn and evolve, they must be continuously monitored to ensure they stay within expected workflows.

How This Works:

AI agents are assigned trust scores based on historical accuracy, safety, and compliance. If the AI starts behaving outside expected parameters, its trust score decreases. If the trust score drops below a threshold, the system requires the AI agent to be re-authenticated before the AI agent is allowed to continue with performing tasks (e.g., accessing additional resources, etc.). In some embodiments, the system may abort the AI agent's current task if the trust score is too low (e.g., drops below another threshold).

Example: A low-risk AI agent that only schedules patient follow-ups has a high trust score. A diagnostic AI suggesting unapproved treatments will be flagged for trust score reduction and additional verification.

Smart Contracts & Blockchain for Workflow Auditing

To prevent AI from bypassing workflow restrictions, all AI actions are recorded in a tamper-proof blockchain ledger.

How This Works:

Each AI agent logs every action (e.g., retrieving patient data, requesting tests) to the blockchain **150**. If an AI agent attempts unauthorized actions, its token is revoked automatically. A hospital administrator can audit all AI agent activities for compliance.

Example: A hospital uses Hyperledger Fabric or Ethereum smart contracts to record who authorized which AI actions. If an AI agent tries to modify patient records without logging an approval, its authentication is immediately revoked. If it suddenly tries to order medications it wasn't programmed for, or access unauthorized patient data, its authentication is also revoked. The AI agent is forced to re-authenticate with the delegator or owner (e.g., the doctor, etc.), and the delegator must manually approve via credentials, such as a biometric scan.

Adaptive AI Access Control

If the AI agent correctly follows medical protocol, its trust score improves, allowing smoother interactions. However, if the AI agent's behavior diverges significantly from historical norms (e.g., ordering high-risk drugs without proper review), the action may trigger immediate lockdown of the AI agent's access by the authentication module **132**. The AI agent must be re-certified before continuing.

Use Case 3: Multi-Agent AI Collaboration for Enterprise Security (Cybersecurity & Network Access)

Scenario: A cybersecurity AI manages access control for a company's internal networks. It collaborates with multiple AI agents to monitor for intrusions, authenticate employees attempting remote access, and issue temporary access credentials for external contractors.

How Dynamic AI Authentication Works:

Step 1: AI Agent Requests Access on Behalf of a User

An IT support AI agent requests remote access to the company's network for an external contractor. The AI agent's identity token (AIT) is verified to ensure that it is an approved IT agent, and it has delegated permission from the security team.

Step 2: AI Agents Authenticate Each Other

The IT AI agent communicates with a Network Security AI agent. Before granting access, the Network Security AI agent requires the IT AI agent to prove its legitimacy using a blockchain-based trust ledger (e.g., the blockchain **150**) to ensure that the IT AI agent is in good standing). The real-time agent verification (e.g., verifying the proof of state of the IT AI agent) also ensures the IT AI agent has not been modified since the last time the IT AI agent was authenticated.

Step 3: Continuous Risk-Based Authentication

The Network Security AI agent monitors the IT AI agent's activity. If the IT AI only grants access to approved users, no further checks are needed. If the IT AI agent tries to grant access to an unrecognized external contractor, it must re-authenticate with the authentication module **132** dynamically using a real-time blockchain identity check, and provide a zero-knowledge proof showing that the contractor has valid permissions. If the authentication fails, the IT AI agent's access is revoked immediately.

Step 4: AI Agent Trust Level Adjustments

If the IT AI agent continuously follows security best practices, it can issue access without frequent verification. However, if the AI shows risky behavior, the authentication module **132** may require stricter authentication steps before granting access to the IT AI agent, and may disable the IT

AI agent entirely if it is determined that the IT AI agent has been comprised (e.g., the code of the IT AI agent has been modified).

FIG. **2** shows an example blockchain network **200** comprising a plurality of interconnected nodes or devices **205***a-h* (generally referred to as nodes **205**). Each of the nodes **205** may comprise a computing device **1000** described in more detail with reference to FIG. **10**. In some embodiments, each of the nodes may correspond to a server that is designated to access and/or maintain the personal profiles of users according to various embodiments of the disclosure. For example, the service provider server **130** may be one of the nodes **205**. In addition, each of the servers (e.g., the data servers **120** and **170**) may also be nodes **205** as part of the blockchain network **200**. The blockchain network **200** may be associated with a blockchain **220**, which may correspond to the blockchain **150**. Some or all of the nodes **205** may replicate and save an identical copy of the blockchain **220**. For example, FIG. **2** shows that the nodes **205***b-e* and **205***g-h* store copies of the blockchain **220**. The nodes **205***b-e* and **205***g-h* may independently update their respective copies of the blockchain **220** as discussed below.

FIG. **3** shows an example blockchain **300**. The blockchain **300** may comprise a plurality of blocks **305***a*, **305***b*, and **305***c* (generally referred to as blocks **305**). The blockchain **300** comprises a first block (not shown), sometimes referred to as the genesis block. Each of the blocks **305** may comprise a record of one or a plurality of submitted and validated transactions. Each of the blocks **305** may comprise one or more data fields. The organization of the blocks **305** within the blockchain **300** and the corresponding data fields may be implementation specific. As an example, the blocks **305** may comprise a respective header **320***a*, **320***b*, and **320***c* (generally referred to as headers **320**) and block data **375***a*, **375***b*, and **375***c* (generally referred to as block data **375**). The headers **320** may comprise metadata associated with their respective blocks **305**. For example, the headers **320** may comprise a respective block number **325***a*, **325***b*, and **325***c*. The headers **320** may also comprise a respective timestamp **350***a*, **350***b*, and **350***c*. As shown in FIG. **3**, the block number **325***a* of the block **305***a* is N-1, the block number **325***b* of the block **305***b* is N, and the block number **325***c* of the block **305***c* is N+1. The headers **320** of the blocks **305** may include a data field comprising a block size (not shown).

The blocks **305** may be linked together and cryptographically secured. For example, the header **320***b* of the block N (block **305***b*) includes a data field (previous block hash **330***b*) comprising a hash representation of the previous block N−1's header **320***a*. The hashing algorithm utilized for generating the hash representation may be, for example, a secure hashing algorithm 256 (SHA-256) which results in an output of a fixed length. In this example, the hashing algorithm is a one-way hash function, where it is computationally difficult to determine the input to the hash function based on the output of the hash function. Additionally, the header **320***c* of the block N+1 (block **305***c*) includes a data field (previous block hash **330***c*) comprising a hash representation of block N's (block **305***b*) header **320***b*. The block **305***a* may also be linked from a previous block header **319**.

The headers **320** of the blocks **305** may also include data fields comprising a hash representation of the block data, such as the block data hash **370***a-c*. The block data hash **370***a-c* may be generated, for example, by a Merkle tree and by storing the hash or by using a hash that is based on all of the block data. The headers **320** of the blocks **305** may comprise a respective nonce **360***a*, **360***b*, and **360***c*. In some implementations, the value of the nonce **360***a-c* is an arbitrary string that is concatenated with (or appended to) the hash of the block. The headers **320** may comprise other data, such as a difficulty target.

The blocks **305** may comprise a respective block data **375***a*, **375***b*, and **375***c* (generally referred to as block data **375**). The block data **375** may comprise a record of validated transactions that have also been integrated into the blockchain **200** via a consensus model (described below). As discussed above, the block data **375** may include a variety of different types of data in addition to validated transactions. Block data **375** may include any data, such as text, audio, video, image, or file, that may be represented digitally and stored electronically.

FIG. **4** illustrates an example ledger **400** according to one embodiment of the disclosure. The ledger **400** may correspond to a blockchain that is maintained and managed by a network (comprising a network of computer nodes **200** as shown in FIG. **2**). When a record is processed by any one of the computer nodes (e.g., the server **120** adding an interaction or transaction of a user to the blockchain **150**, etc.), the record is stored in a block that is added to the ledger **400** by the computer node. To produce the ledger **400**, a single device (e.g., the service provider server **130**, the server **120**, etc.) or a distributed network of devices may operate to agree on a single history of records. Each device in the distributed network operates to collect new records into a block, and then to increment a proof-of work system that includes determining a value that when hashed with the block provides a required number of zero bits.

For example, for a block **402** that includes a plurality of records **402***a*, **402***b*, and up to **402***c*, a device in the distributed network may increment a nonce in the block **402** until a value is found that gives a hash of the block **402** the required number of zero bits. The device may then "chain" the block **402** to the previous block **404** (which may have been "chained" to a previous block, not illustrated, in the same manner).

FIG. **5** is a swim lane diagram showing a data flow **500** among different computer systems for authenticating an AI agent. In this example, an AI agent **580** requests to access one or more resources from a data server **520**. The AI agent **580** may submit a request to the data server **520** for accessing the one or more resources. The submission of the request may trigger the blockchain-based authentication process as disclosed herein. For example, the data server **520** may submit an authentication request to an authentication module **532**, which may correspond to the authentication module **132**. The authentication module **532** may establish a connection with the AI agent **580** to perform the authentication process. Via the connection, the authentication module **532** may request the AI agent **580** to provide credentials, including an identity token associated with the AI agent **580**, a proof of state (e.g., a hash of the binary code associated with the AI agent **580**, etc.), the one or more resources being requested, an identity token of a delegator, and other information. The authentication module **532** may retrieve one or more records from the blockchain **150** based on the identity token of the AI agent **580**. The authentication module **532** may use the information from the blockchain records to verify attributes of the AI agent **580**. For example, the authentication module **532** may determine whether the binary code of the AI agent **580** has been modified since the last time the AI agent **580** was authenticated with the authentication module **532**, whether a trust score of the AI agent **580** exceeds a threshold, whether a valid delegation relationship exists between the AI agent **580** and the delegator associated with the identity token, whether the del-

egator (and the AI agent **580**) has permissions to access the one or more resources, etc. Based on the verification, the authentication module **532** may determine whether to authenticate the AI agent or not **580**. For example, if it is determined that the binary code of the AI agent **580** has been modified, the trust score is below the threshold, no valid delegation relationship between the AI agent **580** and the delegator, or the delegator and/or the AI agent **580** has no permissions to access the one or more resources, the authentication module **532** may not authenticate the AI agent **580**.

On the other hand, if it is determined that the binary code of the AI agent **580** has not been modified, the trust score is above the threshold, a valid delegation relationship exists between the AI agent **580** and the delegator, and the delegator and/or the AI agent **580** has permissions to access the one or more resources, the authentication module **532** may authenticate the AI agent **580**. In this example, the authentication module **532** may authenticate the AI agent **580**, and transmit a notification to the data server **520**. The data server **520** may then grant the AI agent **580** access to the one or more resources. The AI agent **580** may use the one or more resources to perform a task (e.g., generate a prediction output), and provide the prediction output to the data server **520**. In some embodiments, the AI agent **580** and/or the data server **520** may implement one or more computer processes based on the prediction output. The authentication module **532** may update the blockchain **150** based on the result. For example, the authentication module **532** may update the trust score (e.g., increasing the trust score) and store a record of the updated trust score on the blockchain **150**. The authentication module **532** may also record the resources being accessed and the task performed by the AI agent **580** on the blockchain **150**.

FIG. **6** is a swim lane diagram showing another data flow **600** among different computer systems for authenticating an AI agent. Steps 1-4 of the data flow **600** is similar to the steps 1-4 of the data flow **500** of FIG. **5**. Specifically, an AI agent **680** submits a request to the data server **620** for accessing one or more resources. The submission of the request may trigger the blockchain-based authentication process as disclosed herein. The data server **620** may submit an authentication request to an authentication module **632**, which may correspond to the authentication module **132**. The authentication module **632** may establish a connection with the AI agent **680** to perform the authentication process. Via the connection, the authentication module **632** may request the AI agent **680** to provide credentials, including an identity token associated with the AI agent **680**, a proof of state (e.g., a hash of the binary code associated with the AI agent **680**, etc.), the one or more resources being requested, an identity token of a delegator, and other information. The authentication module **632** may retrieve one or more records from the blockchain **150** based on the identity token of the AI agent **680**. The authentication module **632** may use the information from the blockchain records to verify attributes of the AI agent **680**.

Unlike the data flow **500**, in this example, the authentication module **632** does not authenticate the AI agent **680**, possibly due to a determination that the binary code of the AI agent **680** has been modified, the trust score of the AI agent **680** is below the threshold, no valid delegation relationship between the AI agent **680** and the delegator, or the delegator and/or the AI agent **680** has no permissions to access the one or more resources. The authentication module **632** may transmit a notification of authentication denied to the data server **620**. The data server **620** may then deny the AI agent **680** access to the one or more resources. Based on the result, the authentication module **632** may update the blockchain **150**. For example, the authentication module **632** may update the trust score (by reducing the trust score of the AI agent **680**) and store a record of the updated trust score on the blockchain **150**.

FIG. **7** illustrates a process **700** for registering an AI agent according to various embodiments of the disclosure. In some embodiments, at least a portion of the process **700** may be performed by the authentication module **132**, although one or more steps may be performed by one or more of the components/devices/modules/systems described herein. The process **700** begins by receiving (at step **705**) a registration request from an AI agent. For example, when an AI agent (e.g., the AI agent **180**, the AI agent, **190**, etc.) is created and deployed, the owner or the creator of the AI agent may instruct the AI agent to submit a registration request to the authentication module **132** of the service provider server **130**.

Upon receiving the registration request, the authentication module **132** generates (at step **710**), for the AI agent, a decentralized identifier and an identity token based on a binary code associated with the AI agent. For example, the authentication module **132** may generate a unique identifier for the AI agent. The unique identifier may be associated with the AI agent for the duration of the life cycle (e.g., until the AI agent is retired or destroyed, etc.). The authentication module **132** may also mint a token associated with the blockchain **150** for the AI agent. Unlike the unique identifier, the token is associated with the AI agent only in its current state. As such, the token is tied (e.g., associated with) to the current model version of the AI agent, the current owner of the AI agent, the current delegator of the AI agent, etc. Whenever a change to the AI agent (e.g., a change to the binary code such as an upgrade to a different version, a change to the ownership, a change of the roles assigned to AI agent, a change of the delegator, etc.), the AI agent is required to re-register with the authentication module **132** to obtain a new token. The authentication module **132** also records (at step **715**) the decentralized identifier and the identity token on the blockchain **150**.

When it is intended for the AI agent to perform tasks on behalf of a human user, the AI agent and the human user may submit a request to establish a delegation relationship between the AI agent and the human user. The authentication module **132** may request an authorization (e.g., credentials) from the human user, and may establish (at step **720**) a delegation relationship between the AI agent and the human user based on the authorization obtained from the human user. The authentication module **132** stores (at step **725**) the delegation relationship in the blockchain **150**. In some embodiments, the authentication module **132** may also link the identity token to the delegation relationship.

After registering the AI agent and deploying the AI agent, the authentication module **132** monitors (at step **730**) the behavior of the AI agent. For example, the authentication module **132** monitors resources being accessed by the AI agent and tasks being performed by the AI agent. The authentication module **132** may generate a behavior profile for the AI agent based on the resources previously accessed by the AI agent and the task performed by the AI agent. The behavior profile represents a norm for the AI agent, including the types of resources frequently accessed by the AI agent and the types of tasks frequently performed by the AI agent. When the AI agent continues to access resources and perform tasks that are consistent to the behavior profile, the authentication module **132** may increase a trust score associated with the AI agent. On the other hand, if the AI agent

behaves in a way that deviates from the norm by a threshold, the authentication module **132** may reduce the trust score associated with the AI agent. The authentication module **132** then updates (at step **735**) a trust score of the AI agent on the blockchain **150**.

FIG. **8** illustrates a process **800** for authenticating an AI agent according to various embodiments of the disclosure. In some embodiments, at least a portion of the process **800** may be performed by the authentication module **132**, although one or more steps may be performed by one or more of the components/devices/modules/systems described herein. The process **800** begins by receiving (at step **805**) an authentication request for an AI agent to perform a task on behalf of a human user. For example, when an AI agent (e.g., the AI agent **180**, the AI agent, **190**, etc.) requests to access one or more resources from a data server (e.g., the data server). The submission of the request may trigger an automatic execution of the blockchain-based authentication process, which can be performed by the authentication module **132** or a device executing a smart contract associated with the blockchain **150**.

As part of the authentication process, the authentication module **132** or the device executing the smart contract may obtain (at step **810**) an identity token and a digital proof from the AI agent. The authentication module **132** or the device executing the smart contract may traverse the blockchain **150** to retrieve one or more records associated with the AI agent based on the identity token. The one or more records may include attributes associated with the AI agent that is linked to the identity token when the AI agent registered with the authentication module **132**. For example, the one or more records may include the model name, the version number, a hash of the binary code of the AI agent when the AI agent registered with the authentication module **132**, a delegation relationship associated with the AI agent, etc. The authentication module **132** or the device executing the smart contract verifies (at step **815**) a version of the AI agent by comparing the digital proof (e.g., a hash of the current binary code of the AI agent) against the hash value stored in the one or more records on the blockchain **150**. The authentication module **132** or the device executing the smart contract also verifies (at step **820**) a delegation relationship between the AI agent and the human user based on a record on the blockchain. For example, the authentication module **132** or the device executing the smart contract determines whether the human user for whom the AI agent is performing the task, has an existing (registered) relationship with the AI agent based on the one or more records on the blockchain **150**.

The authentication module **132** or the device executing the smart contract then determines (at step **825**) whether to authenticate the AI agent based on the information obtained from the AI agent and the one or more records on the blockchain **150**. For example, the authentication module **132** or the device executing the smart contract may authenticate the AI agent if it is verified that the binary code of the AI agent has not been modified since the last time the AI agent was authenticated based on comparing the digital proof against the hash value stored on a record of the blockchain **150**, and that the AI agent has a registered delegation relationship with the human user. The authentication module **132** or the device executing the smart contract then grants access and monitors (at step **830**) behavior of the AI agent. For example, the authentication module **132** or the device executing the smart contract may transmit an authentication signal to the data server, such that the data server may grant the AI agent access to the one or more resources. The authentication module **132** or the device executing the smart contract may also monitor the AI agent's performance of the task (e.g., which specific resources the AI agent has accessed, what task the AI agent has performed, what output the AI agent has generated, etc.). The authentication module **132** or the device executing the smart contract may determine if the behavior of the AI agent is consistent with the behavior profile associated with the AI agent, and may adjust the trust score based on whether the behavior of the AI agent deviates from the norm specified in the behavior profile.

On the other hand, if it is determined that the binary code of the AI agent has been modified since the last time the AI agent was authenticated based on comparing the digital proof against the hash value stored on a record of the blockchain **150**, or that the AI agent has a registered delegation relationship with the human user, the authentication module **132** or the device executing the smart contract may not authenticate the AI agent. The authentication module **132** or the device executing the smart contract may then deny (at step **835**) the AI agent's access to the one or more resources. The authentication module **132** or the device executing the smart contract then updates (at step **840**) the blockchain based on the authentication result.

FIG. **9** illustrates an example artificial neural network **900** that may be used to implement a machine learning model, such as the AI agent **180** and the AI agent **190**. As shown, the artificial neural network **900** includes three layers—an input layer **902**, a hidden layer **904**, and an output layer **906**. Each of the layers **902**, **904**, and **906** may include one or more nodes (also referred to as "neurons"). For example, the input layer **902** includes nodes **932**, **934**, **936**, **938**, **940**, and **942**, the hidden layer **904** includes nodes **944**, **946**, and **948**, and the output layer **906** includes a node **950**. In this example, each node in a layer is connected to every node in an adjacent layer via edges and an adjustable weight is often associated with each edge. For example, the node **932** in the input layer **902** is connected to all of the nodes **944**, **946**, and **948** in the hidden layer **904**. Similarly, the node **944** in the hidden layer is connected to all of the nodes **932**, **934**, **936**, **938**, **940**, and **942** in the input layer **902** and the node **950** in the output layer **906**. While each node in each layer in this example is fully connected to the nodes in the adjacent layer(s) for illustrative purpose only, it has been contemplated that the nodes in different layers can be connected according to any other neural network topologies as needed for the purpose of performing a corresponding task.

The hidden layer **904** is an intermediate layer between the input layer **902** and the output layer **906** of the artificial neural network **900**. Although only one hidden layer is shown for the artificial neural network **900** for illustrative purpose only, it has been contemplated that the artificial neural network **900** used to implement any one of the computer-based models may include as many hidden layers as necessary. The hidden layer **904** is configured to extract and transform the input data received from the input layer **902** through a series of weighted computations and activation functions.

In this example, the artificial neural network **900** receives a set of inputs and produces an output. Each node in the input layer **902** may correspond to a distinct input. For example, when the artificial neural network **900** is used to implement an AI agent, the nodes in the input layer **902** may correspond to different attributes of a prompt.

In some embodiments, each of the nodes **944**, **946**, and **948** in the hidden layer **904** generates a representation, which may include a mathematical computation (or algorithm) that produces a value based on the input values

received from the nodes **932**, **934**, **936**, **938**, **940**, and **942**. The mathematical computation may include assigning different weights (e.g., node weights, edge weights, etc.) to each of the data values received from the nodes **932**, **934**, **936**, **938**, **940**, and **942**, performing a weighted sum of the inputs according to the weights assigned to each connection (e.g., each edge), and then applying an activation function associated with the respective node (or neuron) to the result. The nodes **944**, **946**, and **948** may include different algorithms (e.g., different activation functions) and/or different weights assigned to the data variables from the nodes **932**, **934**, **936**, **938**, **940**, and **942** such that each of the nodes **944**, **946**, and **948** may produce a different value based on the same input values received from the nodes **932**, **934**, **936**, **938**, **940**, and **942**. The activation function may be the same or different across different layers. Example activation functions include but not limited to Sigmoid, hyperbolic tangent, Rectified Linear Unit (ReLU), Leaky ReLU, Softmax, and/or the like. In this way, after a number of hidden layers, input data received at the input layer **902** is transformed into rather different values indicative data characteristics corresponding to a task that the artificial neural network **900** has been designed to perform.

In some embodiments, the weights that are initially assigned to the input values for each of the nodes **944**, **946**, and **948** may be randomly generated (e.g., using a computer randomizer). The values generated by the nodes **944**, **946**, and **948** may be used by the node **950** in the output layer **906** to produce an output value (e.g., a response to a user query, a prediction, etc.) for the artificial neural network **900**. The number of nodes in the output layer depends on the nature of the task being addressed. For example, in a binary classification problem, the output layer may consist of a single node representing the probability of belonging to one class (as in the example shown in FIG. **9**). In a multi-class classification problem, the output layer may have multiple nodes, each representing the probability of belonging to a specific class. When the artificial neural network **900** is used to implement an AI agent, the output node **950** may be configured to generate content (e.g., a prediction of a medical diagnosis, a determination of a computer network issue, etc.).

In some embodiments, the artificial neural network **900** may be implemented on one or more hardware processors, such as CPUs (central processing units), GPUs (graphics processing units), FPGAs (field-programmable gate arrays), Application-Specific Integrated Circuits (ASICs), dedicated AI accelerators like TPUs (tensor processing units), and specialized hardware accelerators designed specifically for the neural network computations described herein, and/or the like. Example specific hardware for neural network structures may include, but not limited to Google Edge TPU, Deep Learning Accelerator (DLA), NVIDIA AI-focused GPUs, and/or the like. The hardware used to implement the neural network structure is specifically configured based on factors such as the complexity of the neural network, the scale of the tasks (e.g., training time, input data scale, size of training dataset, etc.), and the desired performance.

The artificial neural network **900** may be trained by using training data based on one or more loss functions and one or more hyperparameters. By using the training data to iteratively train the artificial neural network **900** through a feedback mechanism (e.g., comparing an output from the artificial neural network **900** against an expected output, which is also known as the "ground-truth" or "label"), the parameters (e.g., the weights, bias parameters, coefficients in the activation functions, etc.) of the artificial neural network **900** may be adjusted to achieve an objective according to the one or more loss functions and based on the one or more hyperparameters such that an optimal output is produced in the output layer **906** to minimize the loss in the loss functions. Given the loss, the negative gradient of the loss function is computed with respect to each weight of each layer individually. Such negative gradient is computed one layer at a time, iteratively backward from the last layer (e.g., the output layer **906** to the input layer **902** of the artificial neural network **900**). These gradients quantify the sensitivity of the network's output to changes in the parameters. The chain rule of calculus is applied to efficiently calculate these gradients by propagating the gradients backward from the output layer **906** to the input layer **902**.

Parameters of the artificial neural network **900** are updated backwardly from the last layer to the input layer (backpropagating) based on the computed negative gradient using an optimization algorithm to minimize the loss. The backpropagation from the last layer (e.g., the output layer **906**) to the input layer **902** may be conducted for a number of training samples in a number of iterative training epochs. In this way, parameters of the artificial neural network **900** may be gradually updated in a direction to result in a lesser or minimized loss, indicating the artificial neural network **900** has been trained to generate a predicted output value closer to the target output value with improved prediction accuracy. Training may continue until a stopping criterion is met, such as reaching a maximum number of epochs or achieving satisfactory performance on the validation data. At this point, the trained network can be used to make predictions on new, unseen data, such as to predict a frequency of future related transactions.

FIG. **10** is a block diagram of a computer system **1000** suitable for implementing one or more embodiments of the present disclosure, including the service provider server **130**, the data servers **120** and **170**, the devices that execute the AI agents **180** and **190**, and the user device **110**. In various implementations, each of the device **110** may include a mobile cellular phone, a tablet, personal computer (PC), laptop, wearable computing device, etc. adapted for wireless communication, and each of the service provider server **130**, the data servers **120** and **170**, and the devices that execute the AI agents **180** and **190** may include a network computing device, such as a server. Thus, it should be appreciated that the devices/servers **110**, **120**, **130**, **170**, **180**, and **190** may be implemented as the computer system **1000** in a manner as follows.

The computer system **1000** includes a bus **812** or other communication mechanism for communicating information data, signals, and information between various components of the computer system **1000**. The components include an input/output (I/O) component **1004** that processes a user (i.e., sender, recipient, service provider) action, such as selecting keys from a keypad/keyboard, selecting one or more buttons or links, etc., and sends a corresponding signal to the bus **1012**. The I/O component **1004** may also include an output component, such as a display **1002** and a cursor control **1008** (such as a keyboard, keypad, mouse, etc.). The display **1002** may be configured to present a login page for logging into a user account or a checkout page for purchasing an item from a merchant. An optional audio input/output component **1006** may also be included to allow a user to use voice for inputting information by converting audio signals. The audio I/O component **1006** may allow the user to hear audio. A transceiver or network interface **1020** transmits and receives signals between the computer system **1000** and other devices, such as another user device, a merchant

server, or a service provider server via a network **1022**, such as network **160** of FIG. **1**. In one embodiment, the transmission is wireless, although other transmission mediums and methods may also be suitable. A processor **1014**, which can be a micro-controller, digital signal processor (DSP), or other processing component, processes these various signals, such as for display on the computer system **1000** or transmission to other devices via a communication link **1024**. The processor **1014** may also control transmission of information, such as cookies or IP addresses, to other devices.

The components of the computer system **1000** also include a system memory component **1010** (e.g., RAM), a static storage component **1016** (e.g., ROM), and/or a disk drive **1018** (e.g., a solid-state drive, a hard drive). The computer system **1000** performs specific operations by the processor **1014** and other components by executing one or more sequences of instructions contained in the system memory component **1010**. For example, the processor **1014** can perform the cryptocurrency transactions functionalities described herein.

Logic may be encoded in a computer readable medium, which may refer to any medium that participates in providing instructions to the processor **1014** for execution. Such a medium may take many forms, including but not limited to, non-volatile media, volatile media, and transmission media. In various implementations, non-volatile media includes optical or magnetic disks, volatile media includes dynamic memory, such as the system memory component **1010**, and transmission media includes coaxial cables, copper wire, and fiber optics, including wires that comprise the bus **1012**. In one embodiment, the logic is encoded in non-transitory computer readable medium. In one example, transmission media may take the form of acoustic or light waves, such as those generated during radio wave, optical, and infrared data communications.

Some common forms of computer readable media include, for example, floppy disk, flexible disk, hard disk, magnetic tape, any other magnetic medium, CD-ROM, any other optical medium, punch cards, paper tape, any other physical medium with patterns of holes, RAM, PROM, EPROM, FLASH-EPROM, any other memory chip or cartridge, or any other medium from which a computer is adapted to read.

In various embodiments of the present disclosure, execution of instruction sequences to practice the present disclosure may be performed by the computer system **1000**. In various other embodiments of the present disclosure, a plurality of computer systems **1000** coupled by the communication link **1024** to the network (e.g., such as a LAN, WLAN, PTSN, and/or various other wired or wireless networks, including telecommunications, mobile, and cellular phone networks) may perform instruction sequences to practice the present disclosure in coordination with one another.

Where applicable, various embodiments provided by the present disclosure may be implemented using hardware, software, or combinations of hardware and software. Also, where applicable, the various hardware components and/or software components set forth herein may be combined into composite components comprising software, hardware, and/or both without departing from the spirit of the present disclosure. Where applicable, the various hardware components and/or software components set forth herein may be separated into sub-components comprising software, hardware, or both without departing from the scope of the present disclosure. In addition, where applicable, it is contemplated that software components may be implemented as hardware components and vice-versa.

Software in accordance with the present disclosure, such as program code and/or data, may be stored on one or more computer readable mediums. It is also contemplated that software identified herein may be implemented using one or more general purpose or specific purpose computers and/or computer systems, networked and/or otherwise. Where applicable, the ordering of various steps described herein may be changed, combined into composite steps, and/or separated into sub-steps to provide features described herein.

The various features and steps described herein may be implemented as systems comprising one or more memories storing various information described herein and one or more processors coupled to the one or more memories and a network, wherein the one or more processors are operable to perform steps as described herein, as non-transitory machine-readable medium comprising a plurality of machine-readable instructions which, when executed by one or more processors, are adapted to cause the one or more processors to perform a method comprising steps described herein, and methods performed by one or more devices, such as a hardware processor, user device, server, and other devices described herein.

What is claimed is:

1. A system, comprising:

a non-transitory memory; and

one or more hardware processors coupled with the non-transitory memory and configured to read instructions from the non-transitory memory to cause the system to perform operations comprising:

receiving a request for authenticating an artificial intelligence (AI) agent for performing a task, wherein the task is associated with accessing a resource;

obtaining, from the AI agent, an agent identity token associated with the AI agent;

executing a smart contract related to a blockchain of an authentication system based on the agent identity token;

based on the executing the smart contract, (1) accessing parameters associated with a neural network structure of the AI agent and training data used to train the AI agent and (2) generating a digital proof of a state of the AI agent based on delegation metadata representing a delegation relationship between the AI agent and a human user, the parameters, and the training data;

accessing, from the blockchain, a blockchain record comprising information representing a last recorded state of the AI agent based on the agent identity token;

determining whether a machine learning model configuration of the AI agent has been modified based on a comparison between the digital proof and the information;

granting the AI agent access to the resource based on a determination that the machine learning model configuration of the AI agent has not been modified;

monitoring a behavior of the AI agent while accessing the resource; and

in response to determining that the behavior of the AI agent deviates from a historical action pattern of the AI agent by a threshold, recording a transaction on the blockchain that revokes the delegation relationship between the AI agent and the human user.

2. The system of claim 1, wherein the operations further comprise:

determining that the resource is associated with the human user;

accessing, from the blockchain, a second record representing the delegation relationship associated with the AI agent; and

verifying that the delegation relationship is between the AI agent and the human user based on the second record.

3. The system of claim 1, wherein the operations further comprise:

granting the AI agent access to the resource based on determining that the machine learning model configuration of the AI agent has not been modified; and

recording transaction information associated with the task and the digital proof in the blockchain.

4. The system of claim 1, wherein the monitoring the behavior of the AI agent comprises monitoring one or more actions performed by the AI agent using the resource.

5. The system of claim 1, wherein the generating the digital proof comprises:

generating a plurality of hash values representing a neural network architecture of the AI agent, a training dataset signature of the training data, a source code of the AI agent, and the delegation metadata, respectively; and

generating the digital proof based on the plurality of hash values.

6. The system of claim 1, wherein the operations further comprise:

in response to determining that the behavior of the AI agent deviates from the historical action pattern of the AI agent by the threshold, reducing a trust score associated with the AI agent; and

recording the reduced trust score on the blockchain.

7. The system of claim 1, wherein the parameters comprise weights associated with a plurality of nodes in the neural network.

8. The system of claim 1, wherein the task is related to determining a diagnosis of a computer network issue within an internal computer network.

9. The system of claim 8, wherein the operations further comprise:

performing an action to the internal computer network based on the diagnosis.

10. The system of claim 1, wherein the operations further comprise:

extracting, from the request, network information indicating a computer environment in which the AI agent is executed; and

determining a computer network distance between the computer environment and the resource, wherein the determining whether to grant the AI agent access to the resource is further based on the computer network distance.

11. A method, comprising:

receiving, by a computer system, a request for authenticating an artificial intelligence (AI) agent for performing a task, wherein the task is associated with accessing a resource;

obtaining, by the computer system and from the AI agent, an agent identity token associated with the AI agent;

executing a smart contract related to a blockchain of an authentication system based on the agent identity token;

based on the executing the smart contract, (1) accessing parameters associated with a neural network structure of the AI agent and training data used to train the AI agent and (2) generating a digital proof of a state of the AI agent based on delegator metadata representing a delegation relationship between the AI agent and a human user, the parameters, and the training data;

accessing, by the computer system from the blockchain, a blockchain record comprising information representing a last recorded state of the AI agent based on the agent identity token;

determining, by the computer system, whether a machine learning model configuration of the AI agent has been modified based on a comparison between the digital proof and the information;

determining, by the computer system, whether to grant the AI agent access to the resource based on the determining whether the machine learning model configuration of the AI agent has been modified;

monitoring, by the computer system, a behavior of the AI agent while accessing the resource; and

in response to determining that the behavior of the AI agent deviates from a historical action pattern of the AI agent by a threshold, recording, by the computer system, a transaction on the blockchain that revokes the delegation relationship between the AI agent and the human user.

12. The method of claim 11, further comprising:

determining that the resource is associated with the human user;

accessing, from the blockchain, a second record representing the delegation relationship associated with the AI agent; and

verifying that the delegation relationship is between the AI agent and the human user based on the second record.

13. The method of claim 11, further comprising:

granting the AI agent access to the resource based on determining that the machine learning model configuration of the AI agent has not been modified; and

recording transaction information associated with the task and the digital proof in the blockchain.

14. The method of claim 11,

wherein the monitoring the behavior of the AI agent comprises monitoring one or more actions performed by the AI agent using the resource.

15. The method of claim 11, wherein the generating the digital proof comprises:

generating a plurality of hash values representing a neural network architecture of the AI agent, a training dataset signature of the training data, a source code of the AI agent, and the delegation metadata, respectively; and

generating the digital proof based on the plurality of hash values.

16. The method of claim 11, further comprising:

in response to determining that the behavior of the AI agent deviates from the historical action pattern of the AI agent by the threshold, reducing a trust score associated with the AI agent; and

recording the reduced trust score on the blockchain.

17. The method of claim 11, wherein the parameters comprise weights associated with a plurality of nodes in the neural network.

18. The method of claim 11, wherein the task is related to determining a diagnosis of a computer network issue within an internal computer network.

19. The method of claim 18, further comprising:

performing an action to the internal computer network based on the diagnosis.

20. The method of claim 11, further comprising:

extracting, from the request, network information indicating a computer environment in which the AI agent is executed; and

determining a computer network distance between the computer environment and the resource, wherein the determining whether to grant the AI agent access to the resource is further based on the computer network distance.

* * * * *